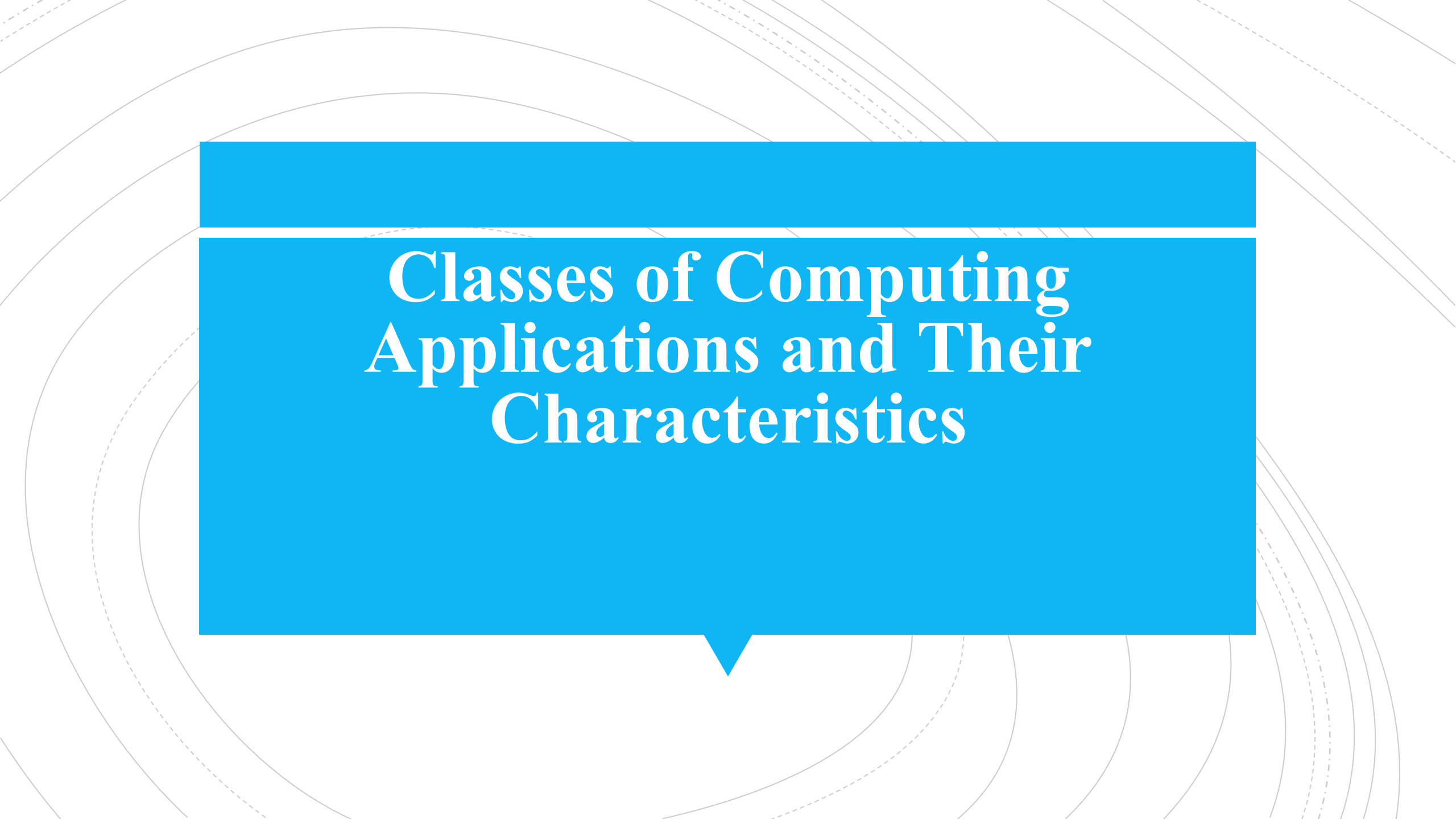


Computer Architecture

Module III

Module 3

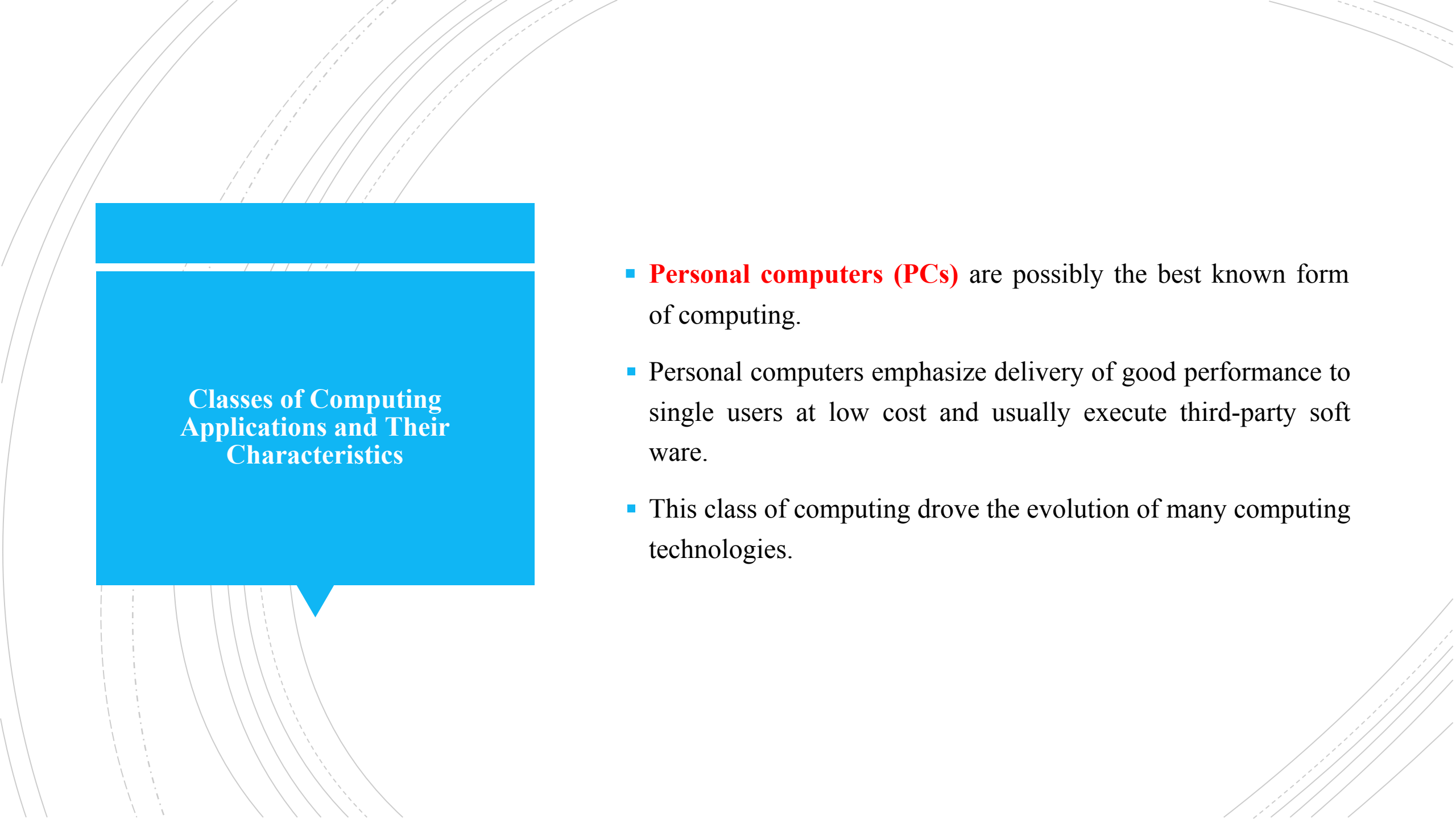
Computer abstractions and technology -Introduction,Computer architecture -8 Design features,Application program - layers of abstraction, Five key components of a computer, Technologies for building processors and memory, Performance, Instruction set principles – Introduction,Classifying instruction set architectures, Memory addressing, Encoding an instruction set.

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid blue speech bubble is centered on the page, pointing downwards. The title text is contained within this bubble.

Classes of Computing Applications and Their Characteristics

What is a computer ?

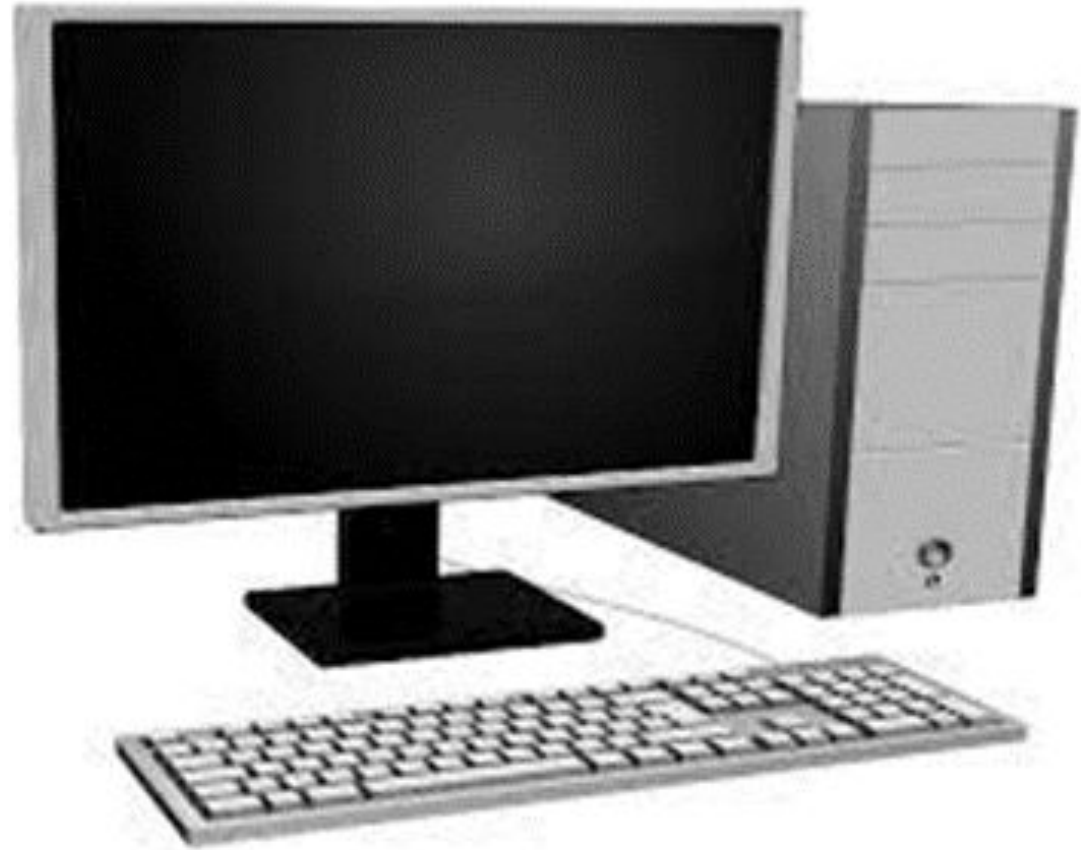
- A **computer** is an electronic device that manipulates information, or data. It has the ability to **store**, **retrieve**, and **process** data.

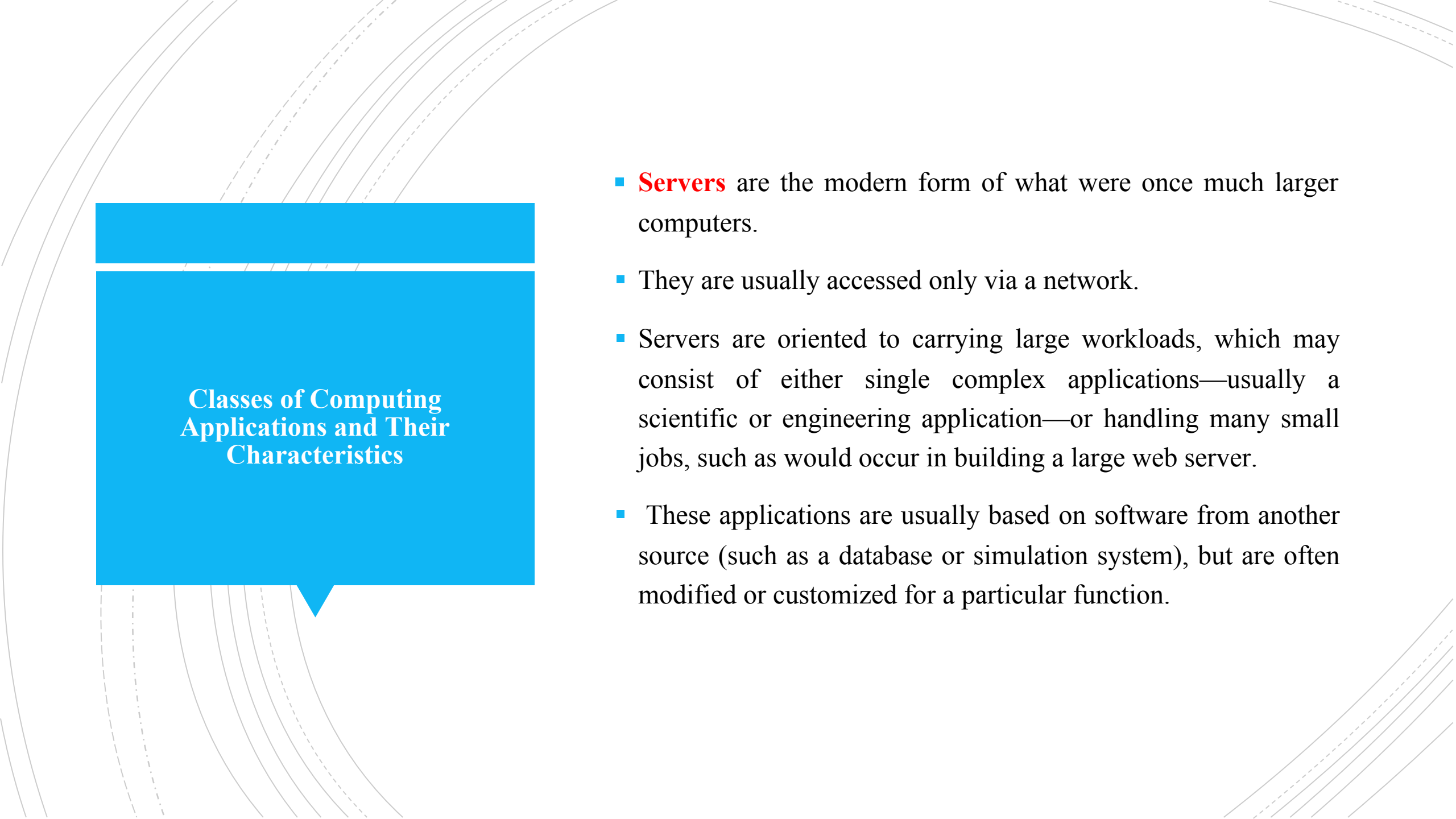
The background of the slide features a series of concentric, curved lines in a light gray color, creating a sense of motion or a stylized globe. These lines are more prominent on the left side and fade out towards the right.

Classes of Computing Applications and Their Characteristics

- **Personal computers (PCs)** are possibly the best known form of computing.
- Personal computers emphasize delivery of good performance to single users at low cost and usually execute third-party software.
- This class of computing drove the evolution of many computing technologies.

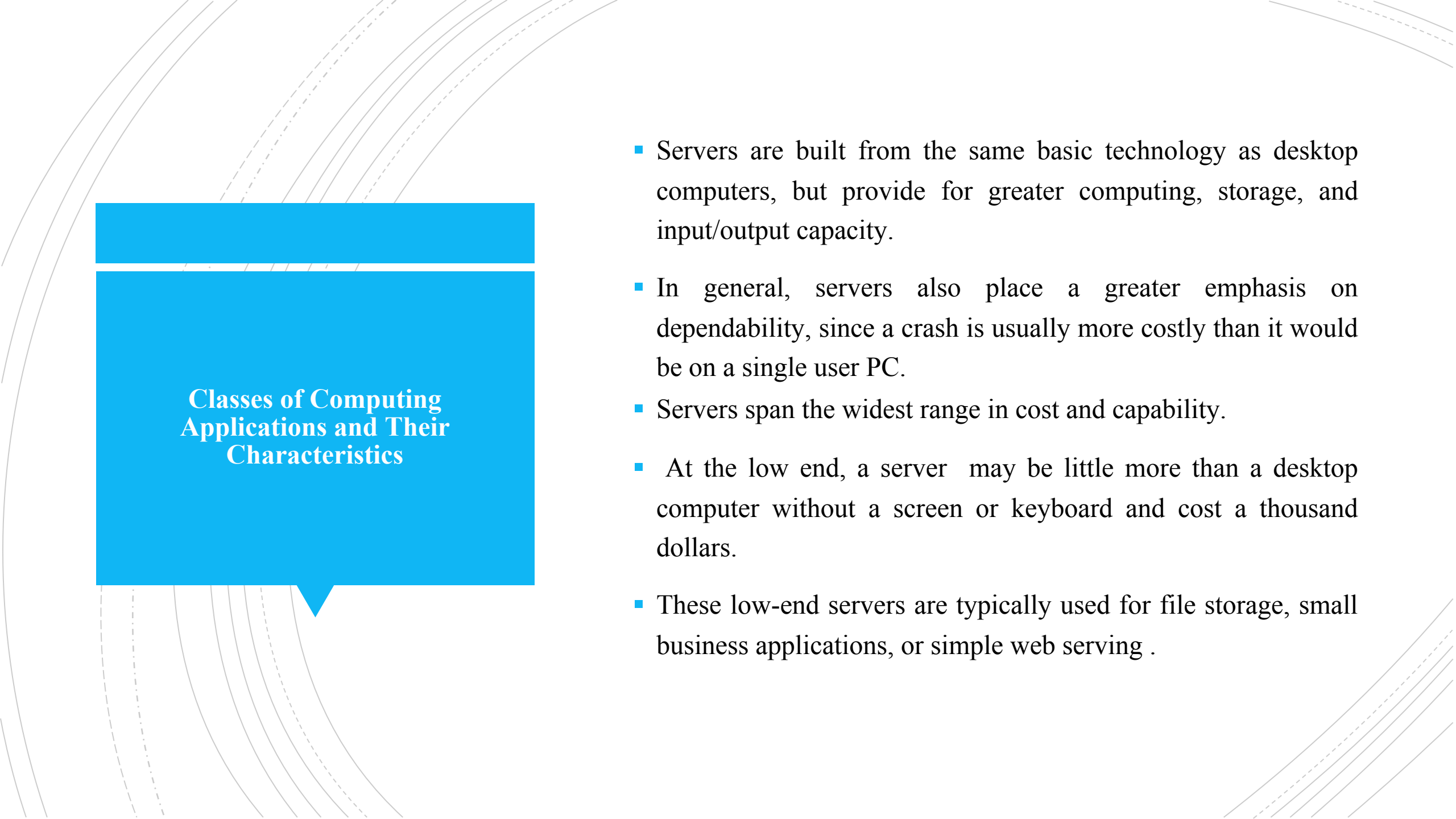
Personal computers



The background of the slide features a series of concentric, curved lines in a light gray color, creating a sense of motion or a stylized globe. These lines are more prominent on the left side and fade out towards the right.

Classes of Computing Applications and Their Characteristics

- **Servers** are the modern form of what were once much larger computers.
- They are usually accessed only via a network.
- Servers are oriented to carrying large workloads, which may consist of either single complex applications—usually a scientific or engineering application—or handling many small jobs, such as would occur in building a large web server.
- These applications are usually based on software from another source (such as a database or simulation system), but are often modified or customized for a particular function.

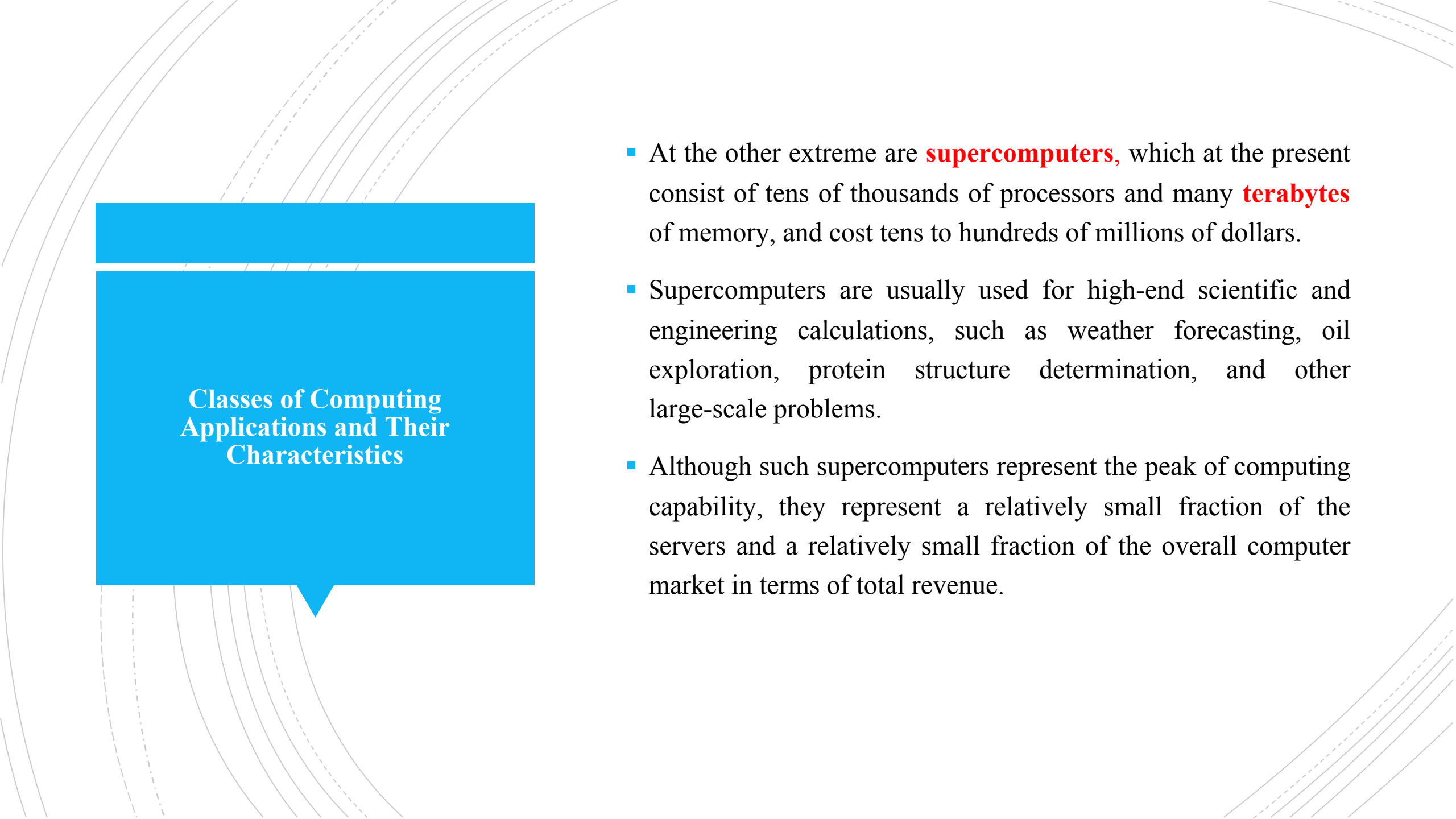


Classes of Computing Applications and Their Characteristics

- Servers are built from the same basic technology as desktop computers, but provide for greater computing, storage, and input/output capacity.
- In general, servers also place a greater emphasis on dependability, since a crash is usually more costly than it would be on a single user PC.
- Servers span the widest range in cost and capability.
- At the low end, a server may be little more than a desktop computer without a screen or keyboard and cost a thousand dollars.
- These low-end servers are typically used for file storage, small business applications, or simple web serving .

Server



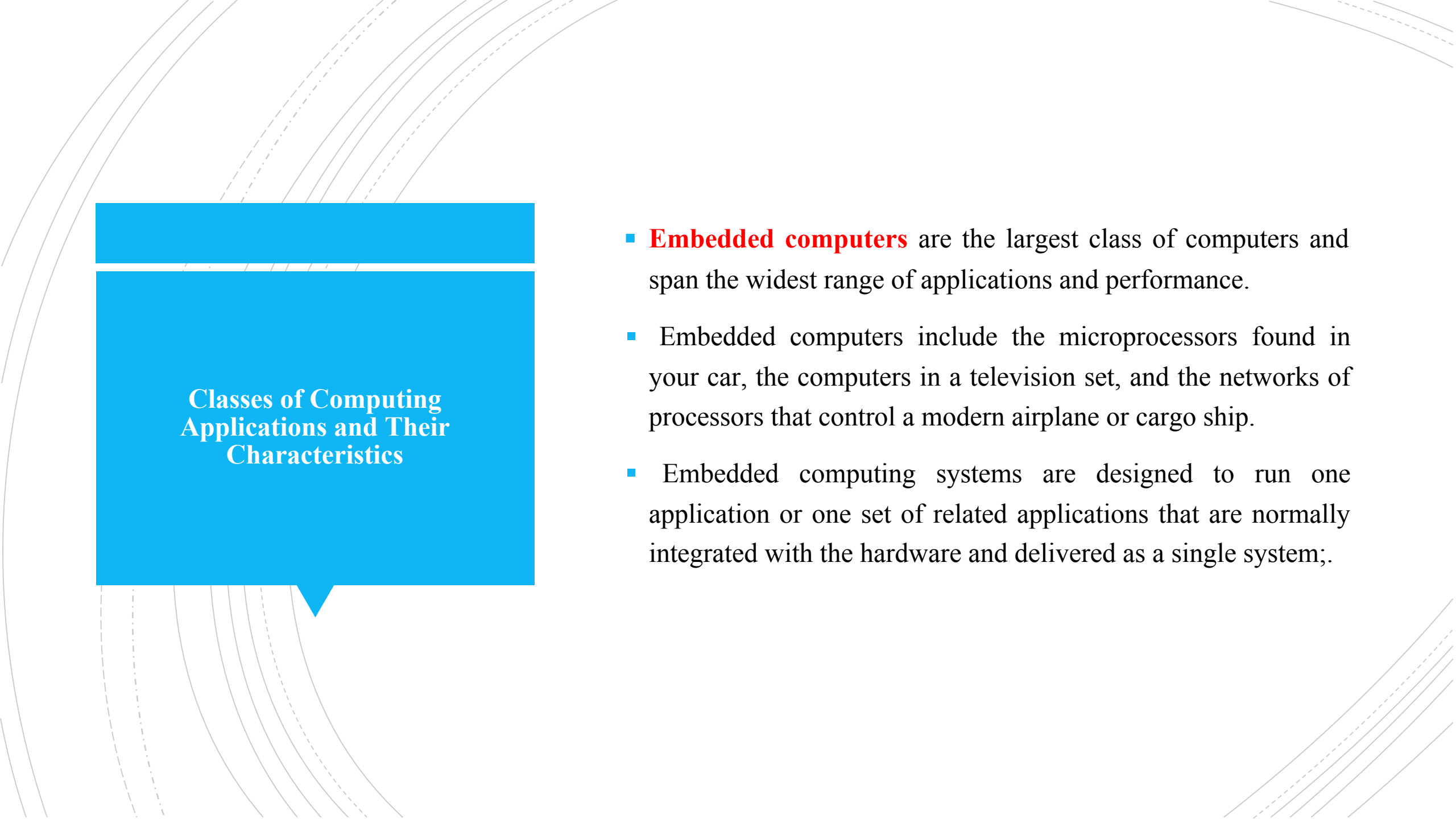
The background of the slide features a series of thin, curved lines in light gray and white, creating a sense of motion and depth. These lines are more prominent on the left side and fade towards the right.

Classes of Computing Applications and Their Characteristics

- At the other extreme are **supercomputers**, which at the present consist of tens of thousands of processors and many **terabytes** of memory, and cost tens to hundreds of millions of dollars.
- Supercomputers are usually used for high-end scientific and engineering calculations, such as weather forecasting, oil exploration, protein structure determination, and other large-scale problems.
- Although such supercomputers represent the peak of computing capability, they represent a relatively small fraction of the servers and a relatively small fraction of the overall computer market in terms of total revenue.

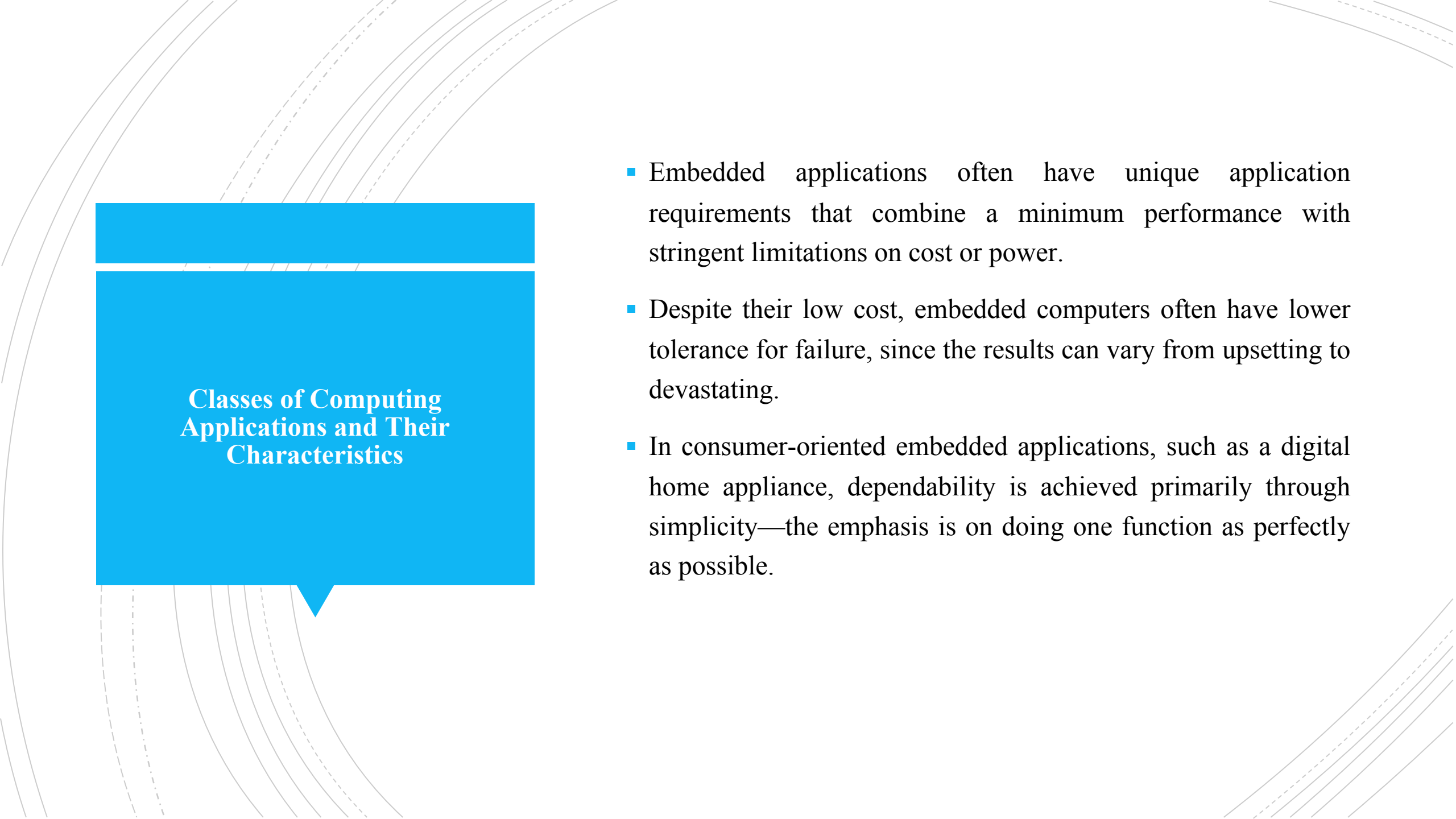
Super computers



The background of the slide features a series of concentric, curved lines in a light gray color, creating a sense of motion or a stylized globe. These lines are more prominent on the left side and fade towards the right.

Classes of Computing Applications and Their Characteristics

- **Embedded computers** are the largest class of computers and span the widest range of applications and performance.
- Embedded computers include the microprocessors found in your car, the computers in a television set, and the networks of processors that control a modern airplane or cargo ship.
- Embedded computing systems are designed to run one application or one set of related applications that are normally integrated with the hardware and delivered as a single system;.

The background of the slide features a series of concentric, curved lines in a light gray color, creating a sense of depth and movement. These lines are more prominent on the left side and fade towards the right.

Classes of Computing Applications and Their Characteristics

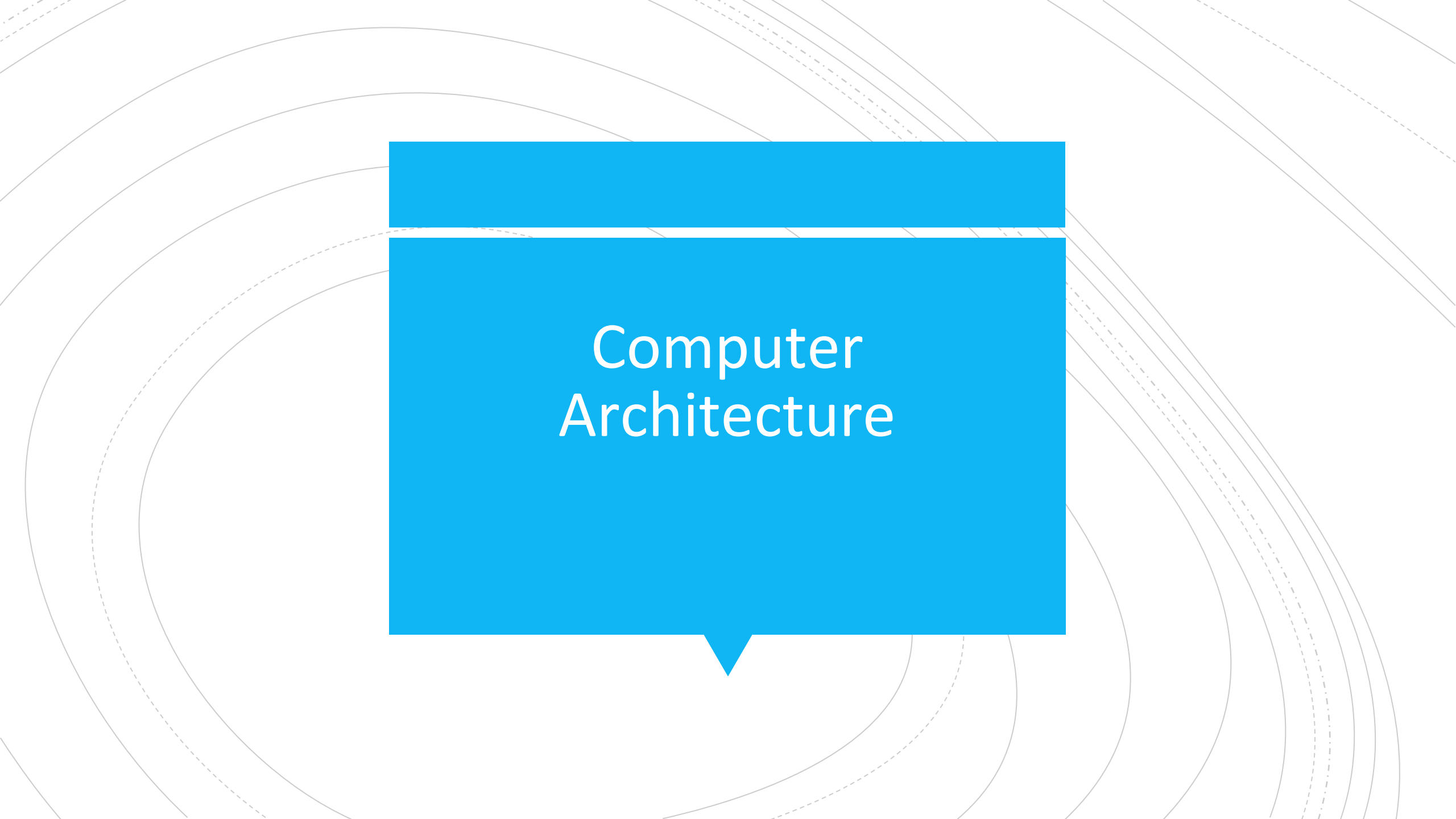
- Embedded applications often have unique application requirements that combine a minimum performance with stringent limitations on cost or power.
- Despite their low cost, embedded computers often have lower tolerance for failure, since the results can vary from upsetting to devastating.
- In consumer-oriented embedded applications, such as a digital home appliance, dependability is achieved primarily through simplicity—the emphasis is on doing one function as perfectly as possible.

Examples of Embedded Systems

Embedded
computers



Many Different Products Depend on Embedded Systems

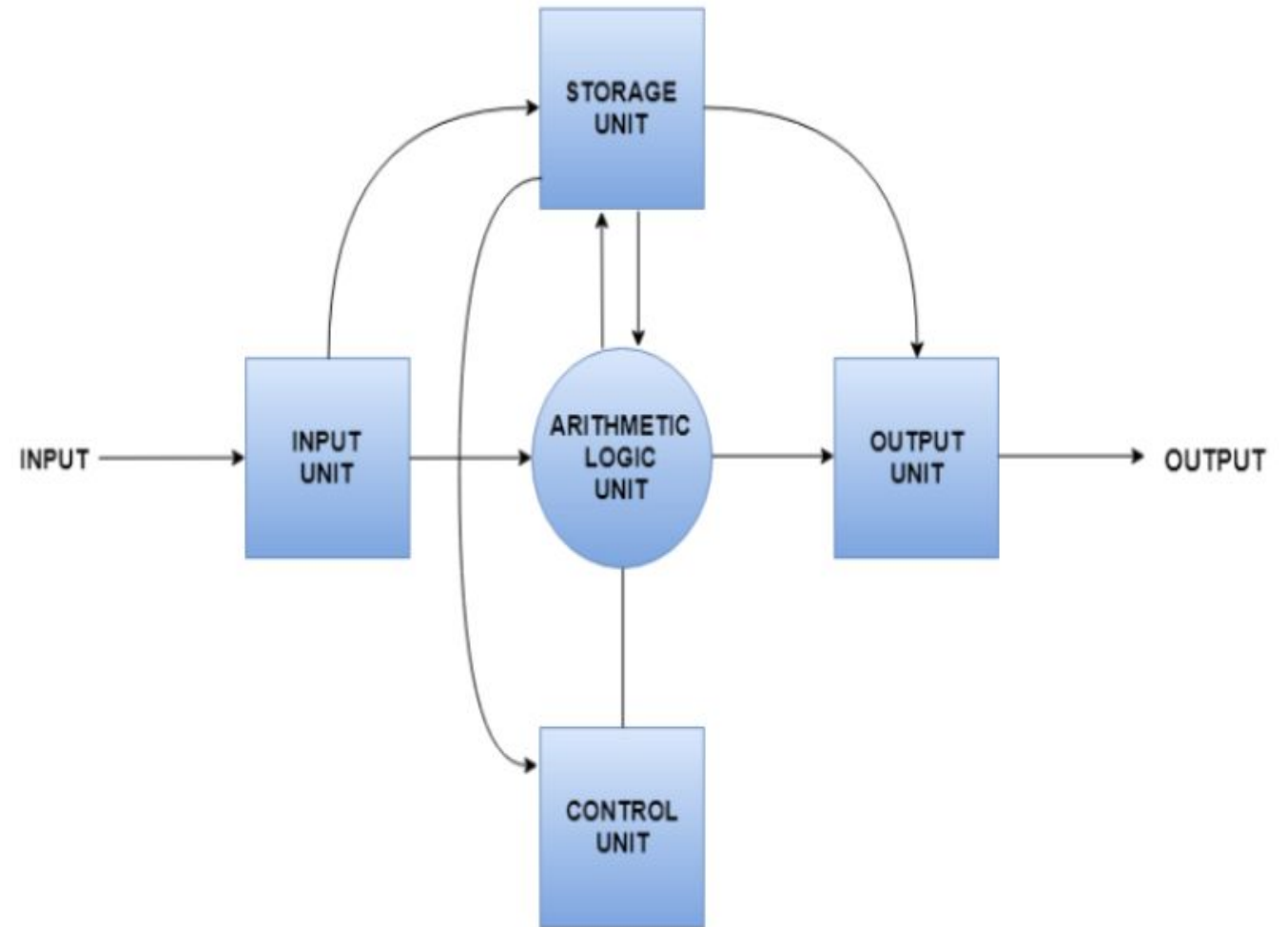
The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A bright blue speech bubble is centered on the page, with a small tail pointing downwards.

Computer Architecture

Computer Architecture

- A computer system is basically a machine that simplifies complicated tasks.
- It should maximize performance and reduce costs as well as power consumption.
- The different components in the Computer System Architecture are
 - Input Unit,
 - Output Unit,
 - Storage Unit,
 - Arithmetic Logic Unit,
 - Control Unit etc.

Computer Architecture



The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large blue speech bubble is centered on the page, containing the title text.

Eight Great Ideas in Computer Architecture

1. Design for Moore's Law

- The one constant for computer designers is rapid change, which is driven largely by **Moore's Law**.
- It states that integrated circuit resources double every 18–24 months.
- Moore's Law resulted from a 1965 prediction of such growth in IC capacity made by Gordon Moore, one of the founders of Intel.
- As computer designs can take years, the resources available per chip can easily double or quadruple between the start and finish of the project.
- The computer architects must anticipate where the technology will be when the design finishes rather than design for where it starts.

2. Use Abstraction to Simplify Design

- Both computer architects and programmers had to invent techniques to make themselves more productive, for otherwise design time would lengthen as dramatically as resources grew by Moore's Law.
- A major productivity technique for hardware and software is to use **abstractions** to represent the design at different levels of representation; lower-level details are hidden to offer a simpler model at higher levels.

3. Make the Common Case Fast

- Making the **common case fast** will tend to enhance performance better than optimizing the rare case.
- Ironically, the common case is often simpler than the rare case and hence is often easier to enhance.
- This common sense advice implies that you know what the common case is, which is only possible with careful experimentation and measurement.

4. Performance via Parallelism

- Since the dawn of computing, computer architects have offered designs that get more performance by performing operations in parallel.
- *parallel computer* is a set of processors that are able to work cooperatively to solve a computational problem.

5. Performance via Pipelining

- A particular pattern of parallelism is so prevalent in computer architecture that it merits its own name: **pipelining**.
- Pipelining is a process of arrangement of hardware elements of the CPU such that its overall performance is increased.
- Simultaneous execution of more than one instruction takes place in a pipelined processor.

6. Performance via Prediction

- In some cases it can be faster on average to guess and start working rather than wait until you know for sure, assuming that the mechanism to recover from a misprediction is not too expensive and your prediction is relatively accurate.

7. Hierarchy of Memories

- Programmers want memory to be fast, large, and cheap, as memory speed often shapes performance, capacity limits the size of problems that can be solved, and the cost of memory today is often the majority of computer cost.
- Architects have found that they can address these conflicting demands with a **hierarchy of memories**, with the fastest, smallest, and most expensive memory per bit at the top of the hierarchy and the slowest, largest, and cheapest per bit at the bottom.

8. Dependability via Redundancy

- Computers not only need to be fast; they need to be dependable.
- Since any physical device can fail, we make systems **dependable** by including redundant components that can take over when a failure occurs and to help detect failures.

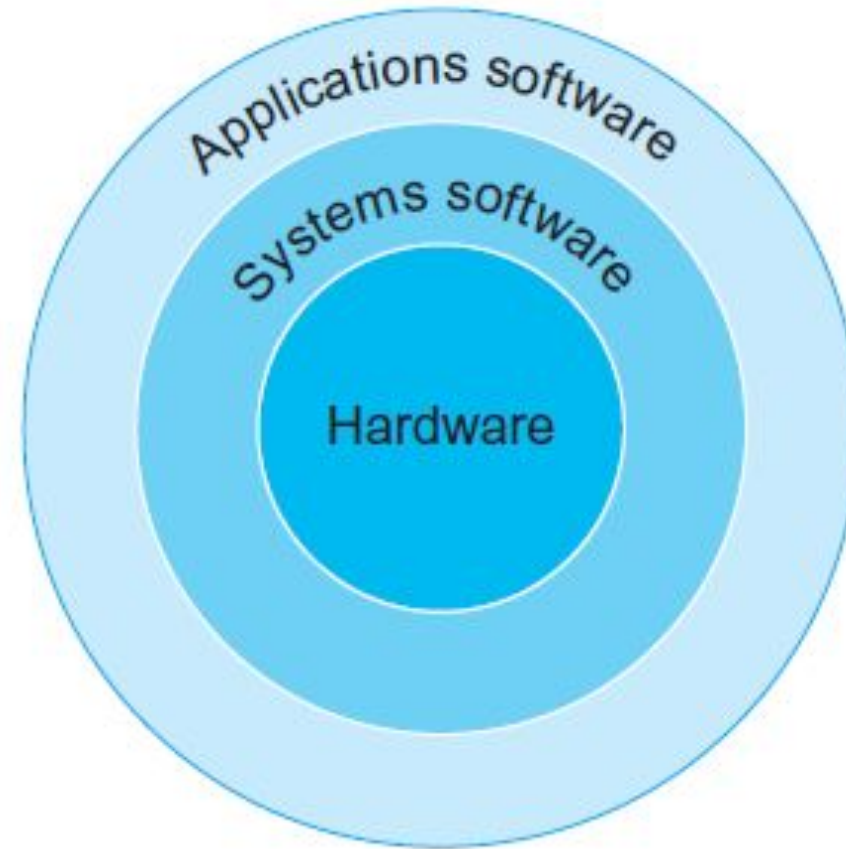
The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid blue speech bubble is centered on the page, pointing downwards.

Application Programs

Application Program

- A typical application, such as a word processor or a large database system, may consist of millions of lines of code and rely on sophisticated software libraries that implement complex functions in support of the application.
- The hardware in a computer can only execute extremely simple low-level instructions.
- To go from a complex application to the simple instructions involves several layers of software that interpret or translate high-level operations into simple computer instructions, an example of the great idea of **abstraction**.

Application Program





Application Program

- **Figure** shows that these layers of software are organized primarily in a hierarchical fashion, with applications being the outermost ring and a variety of **systems software** sitting between the hardware and applications software.

Systems software

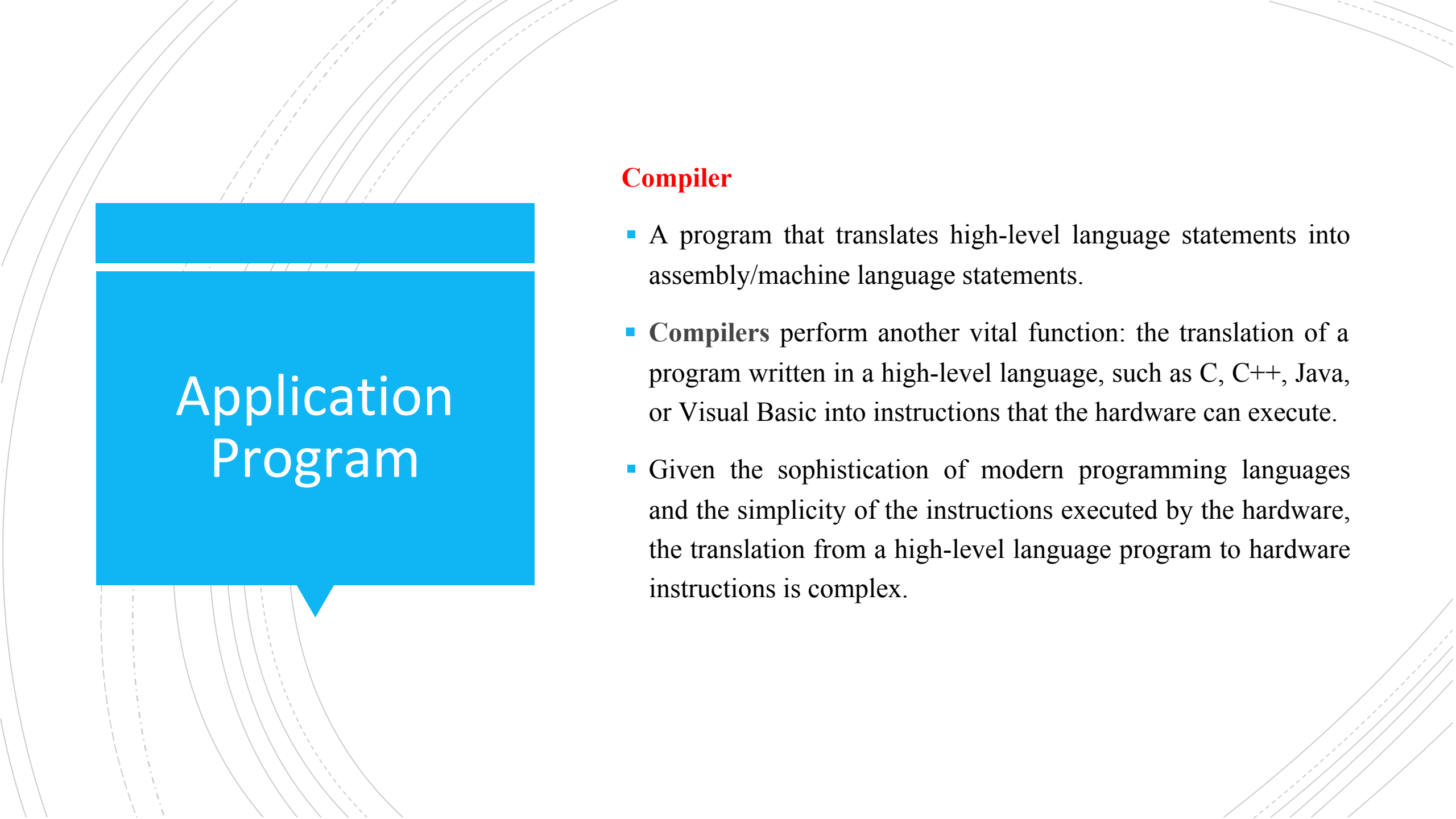
- Software that provides services that are commonly useful, including operating systems, compilers, loaders, and assemblers.
- There are many types of systems software, but two types of systems software are central to every computer system today: an operating system and a compiler.



Application Program

Operating system

- Supervising program that manages the resources of a computer for the benefit of the programs that run on that computer. An **operating system** interfaces between a user's program and the hardware and provides a variety of services and supervisory functions. Among the most important functions are:
 - Handling basic input and output operations
 - Allocating storage and memory
 - Providing for protected sharing of the computer among multiple applications using it simultaneously.
- Examples of operating systems in use today are Linux, iOS, and Windows.

The background of the slide features a series of concentric, curved lines in a light gray color, creating a sense of motion or a stylized globe. These lines are more prominent on the left side and fade towards the right.

Application Program

Compiler

- A program that translates high-level language statements into assembly/machine language statements.
- **Compilers** perform another vital function: the translation of a program written in a high-level language, such as C, C++, Java, or Visual Basic into instructions that the hardware can execute.
- Given the sophistication of modern programming languages and the simplicity of the instructions executed by the hardware, the translation from a high-level language program to hardware instructions is complex.

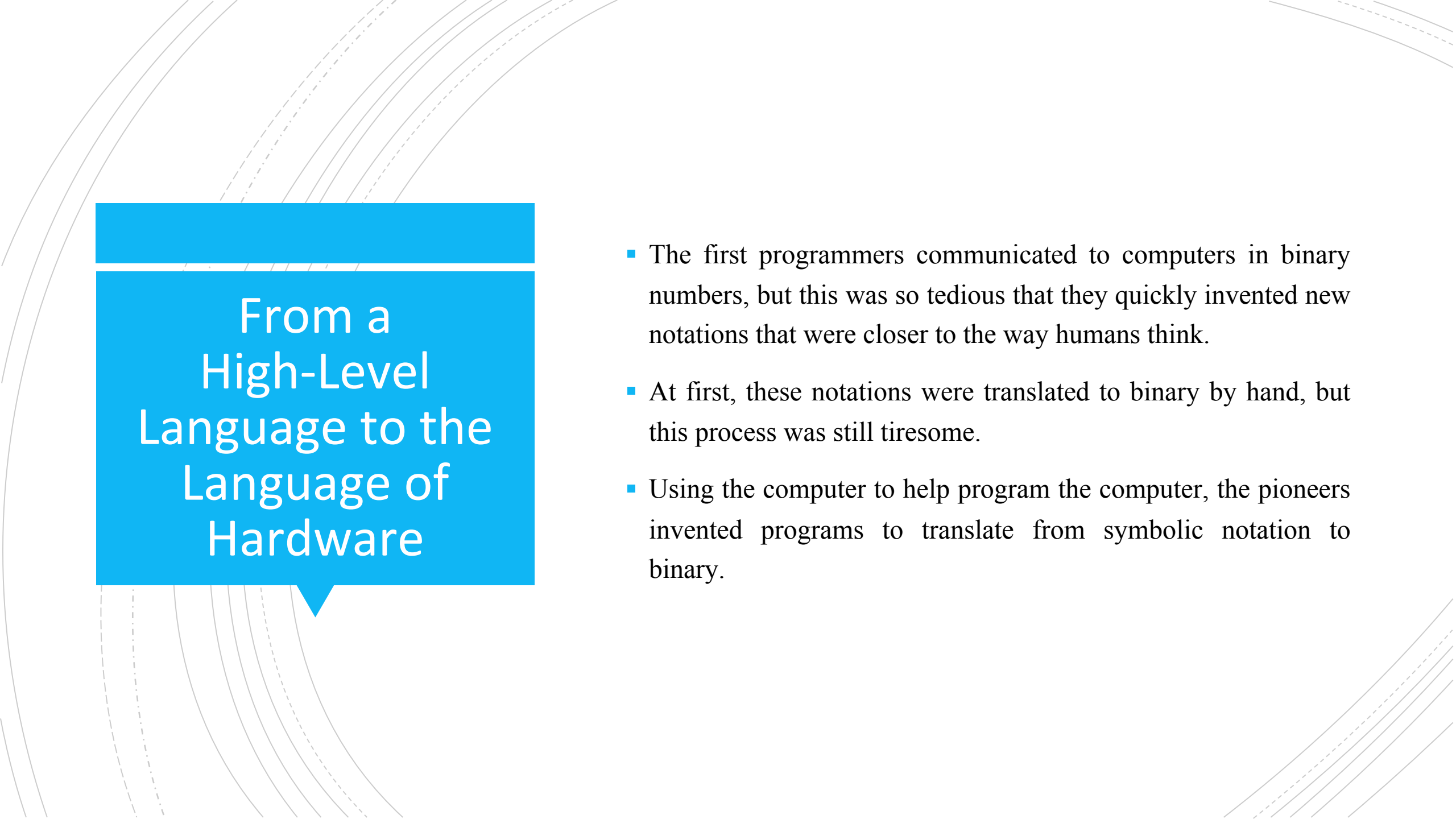
From a High-Level Language to the Language of Hardware

- The easiest signals for computers to understand are *on* and *off*, and so the computer alphabet is just two letters.
- The two symbols for these two letters are the numbers 0 and 1, and we commonly think of the computer language as numbers in base 2, or *binary numbers*.
- We refer to each “letter” as a **binary digit or bit**.
- Computers are slaves to our commands, which are called **instructions**.

From a High-Level Language to the Language of Hardware

Instruction

- A command that computer hardware understands and obeys.
- Instructions, which are just collections of bits that the computer understands and obeys, can be thought of as numbers.
- For example, the bits `1000110010100000` tell one computer to add two numbers.

The background of the slide features a series of thin, curved lines in shades of gray, creating a sense of motion and depth. These lines are more prominent on the left side and fade towards the right.

From a High-Level Language to the Language of Hardware

- The first programmers communicated to computers in binary numbers, but this was so tedious that they quickly invented new notations that were closer to the way humans think.
- At first, these notations were translated to binary by hand, but this process was still tiresome.
- Using the computer to help program the computer, the pioneers invented programs to translate from symbolic notation to binary.

From a High-Level Language to the Language of Hardware

- The first of these programs was named an **assembler**.
- This program translates a symbolic version of an instruction into the binary version.
- For example, the programmer would write
`add A,B`
- and the assembler would translate this notation into
`1000110010100000`
- This instruction tells the computer to add the two numbers A and B.

Assembler

Assembler

- A program that translates a symbolic version of instructions into the binary version.

Assembly language

- A symbolic representation of machine instructions.

Machine language

- A binary representation of machine instructions.

From a High-Level Language to the Language of Hardware

- Although a tremendous improvement, assembly language is still far from the notations a scientist might like to use to simulate fluid flow or that an accountant might use to balance the books.
- Assembly language requires the programmer to write one line for every instruction that the computer will follow, forcing the programmer to think like the computer.
- The recognition that a program could be written to translate a more powerful language into computer instructions was one of the great breakthroughs in the early days of computing.
- Programmers today owe their productivity to the creation of **high-level programming languages** and compilers that translate programs in such languages into instructions.

From a High-Level Language to the Language of Hardware

High-level
language
program
(in C)

```
swap(int v[], int k)
{int temp;
  temp = v[k];
  v[k] = v[k+1];
  v[k+1] = temp;
}
```

Compiler

Assembly
language
program
(for MIPS)

```
swap:
  multi $2, $5, 4
  add   $2, $4, $2
  lw    $15, 0($2)
  lw    $16, 4($2)
  sw    $16, 0($2)
  sw    $15, 4($2)
  jr    $31
```

Assembler

Binary machine
language
program
(for MIPS)

```
000000001010001000000000100011000
00000000100000100001000000100001
10001101111000100000000000000000
100011100001001000000000000000100
101011100001001000000000000000000
101011011110001000000000000000100
000000111110000000000000000001000
```

From a High-Level Language to the Language of Hardware

- A compiler enables a programmer to write this high-level language expression:

$A + B$

- The compiler would compile it into this assembly language statement:

add A,B

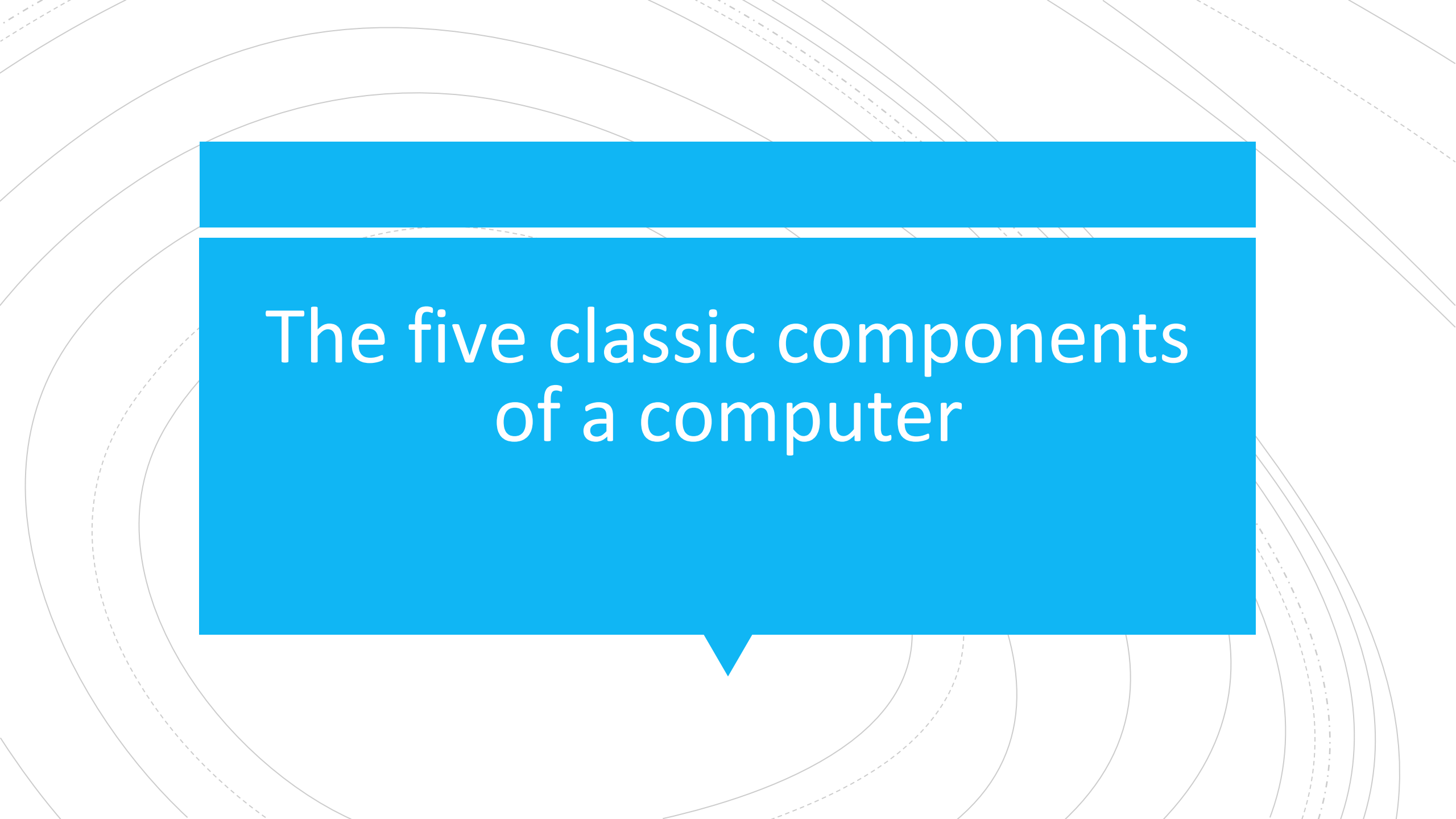
- The assembler would translate this statement into the binary instructions that tell the computer to add the two numbers A and B.

From a High-Level Language to the Language of Hardware

- High-level programming languages offer several important benefits.
- First, they allow the programmer to think in a **more natural language**, using English words and algebraic notation, resulting in programs that look much more like text.
- Moreover, they allow languages to be designed according to their **intended use**.
- Hence, Fortran was designed for scientific computation, Cobol for business data processing, Lisp for symbol manipulation and so on.

From a High-Level Language to the Language of Hardware

- The second advantage of programming languages is improved programmer **productivity**.
- One of the few areas of widespread agreement in software development is that it takes less time to develop programs when they are written in languages that require fewer lines to express an idea.
- **Conciseness** is a clear advantage of high level languages over assembly language.
- The final advantage is that programming languages allow programs to be **independent** of the computer on which they were developed, since compilers and assemblers can translate high-level language programs to the binary instructions of any computer.
- These advantages are so strong that today little programming is done in assembly language.

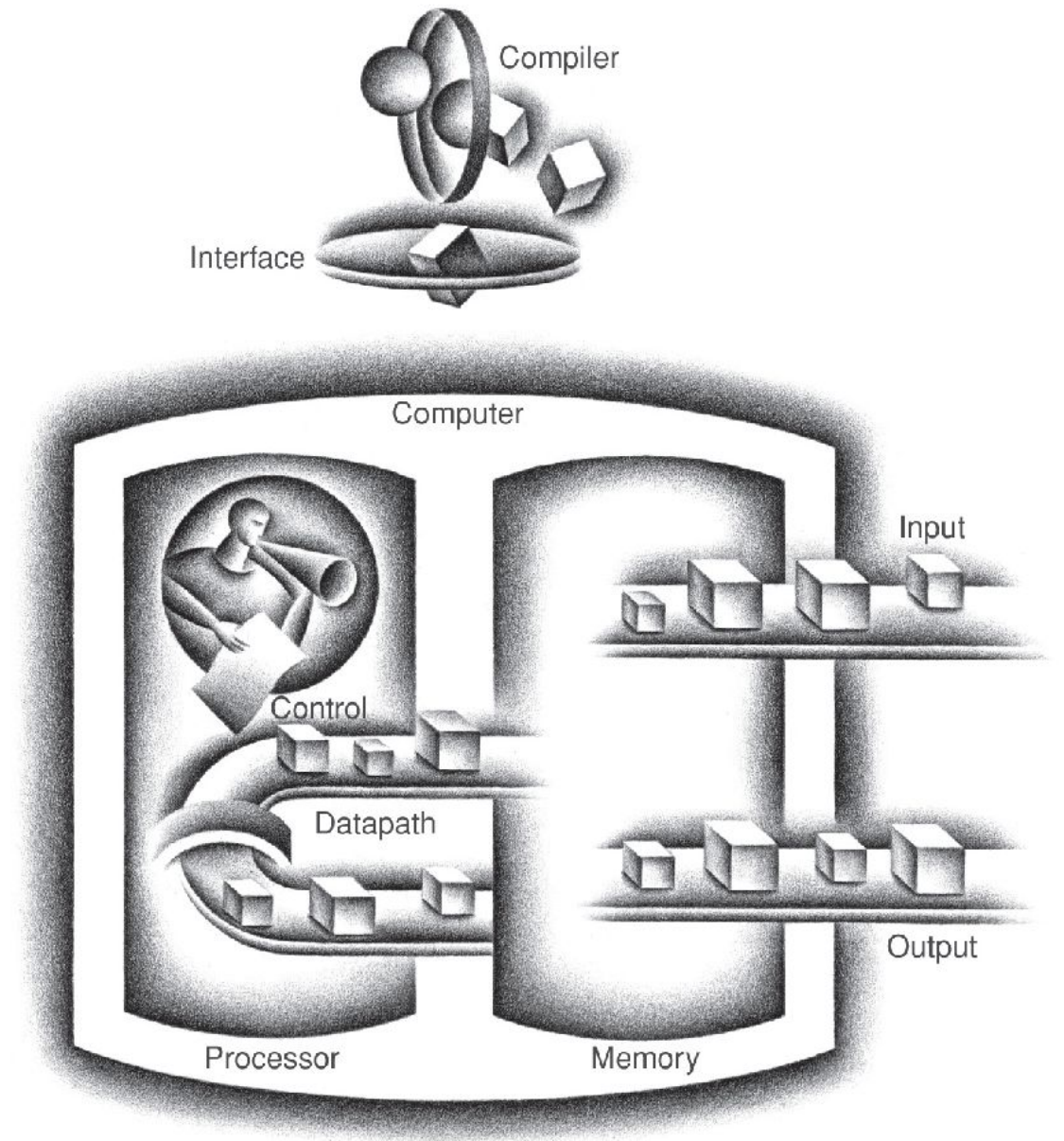
The background of the slide features a series of concentric circles in light gray, some solid and some dashed, creating a subtle pattern. A large, solid blue speech bubble is centered on the slide, containing the title text in white. The speech bubble has a small tail pointing downwards.

The five classic components of a computer

The five classic components of a computer

- The five classic components of a computer are
 - Input
 - Output
 - Memory
 - Data path
 - Control
- With the last two sometimes combined and called the processor.

The five classic components of a computer



The five classic components of a computer

Input device

- A mechanism through which the computer is fed information, such as a keyboard.

Output device

- A mechanism that conveys the result of a computation to a user, such as a display, or to another computer.

Memory

- It refers to a device that is used to store information for immediate use in a computer or related computer hardware device.

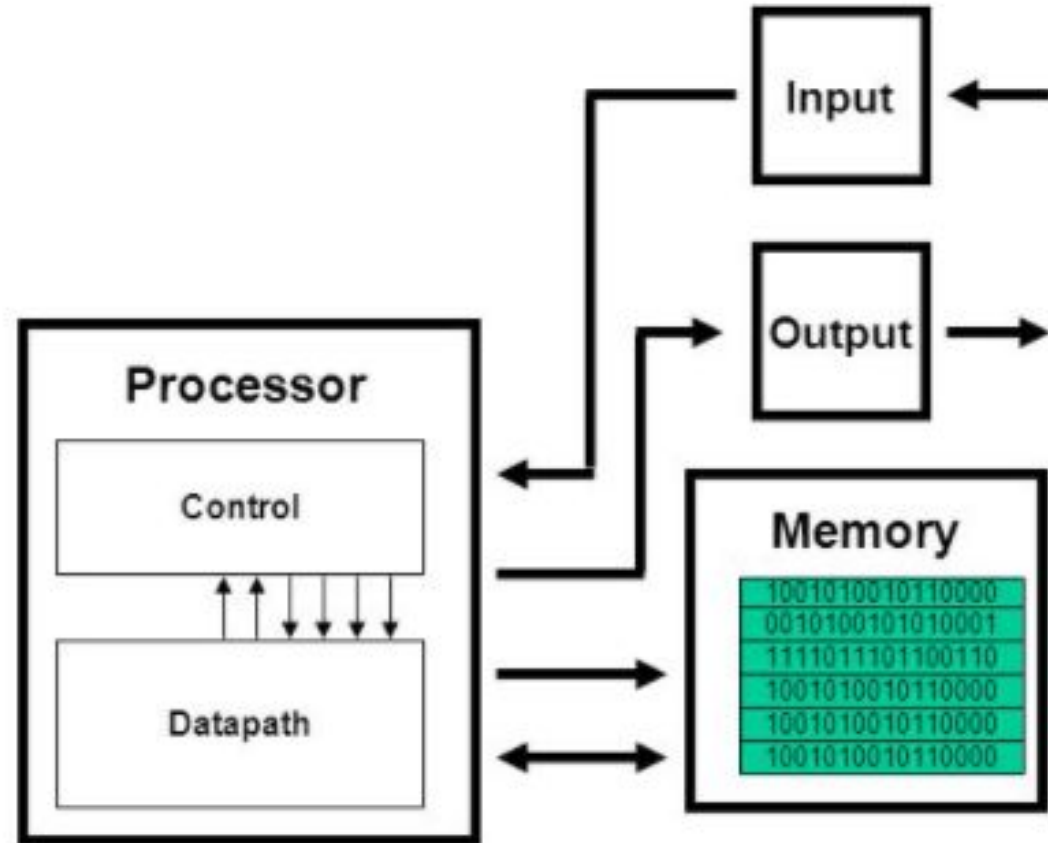
Control

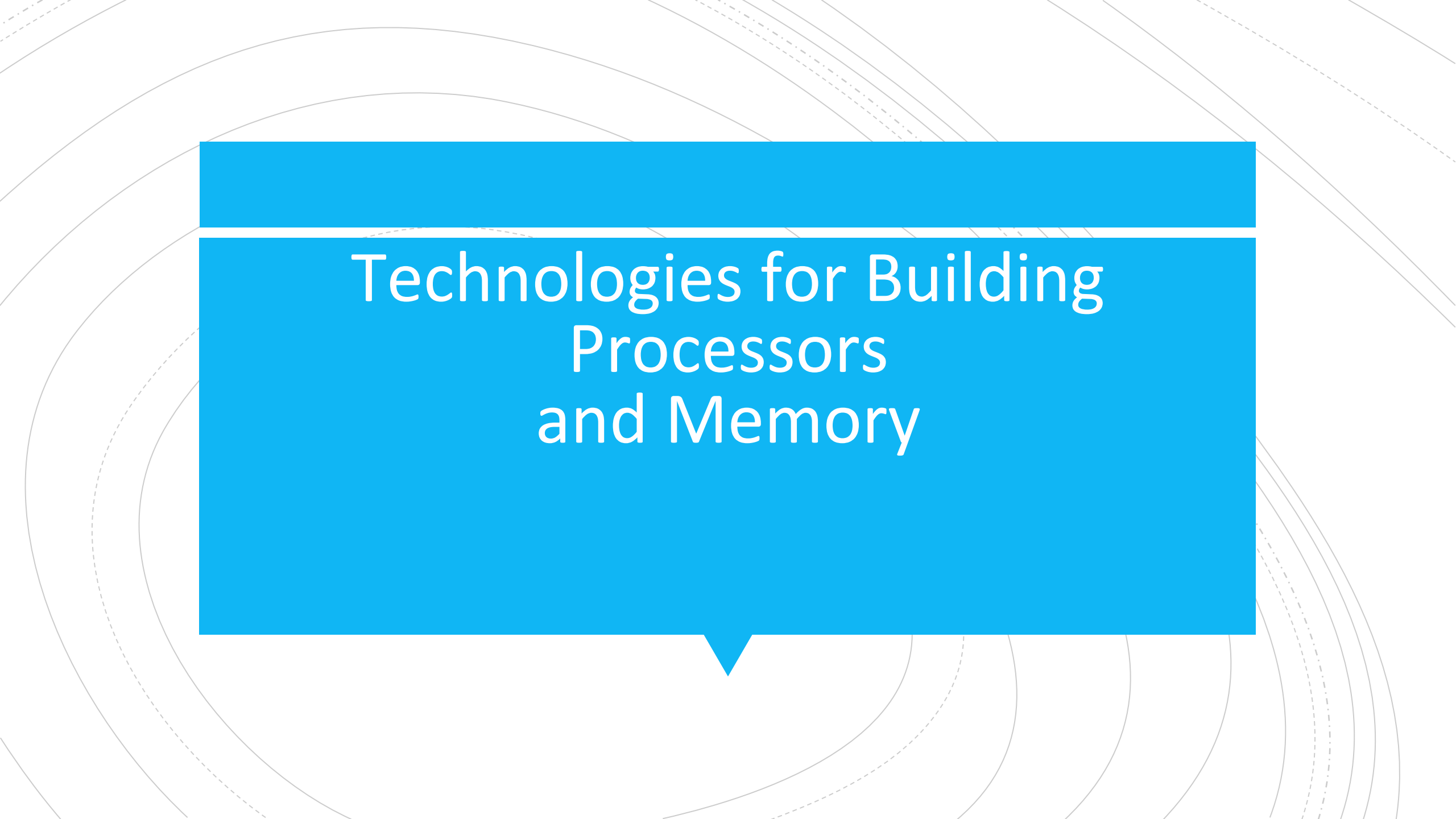
- Control unit of the central processing unit regulates and integrates the operations of the computer.

Data path

- A data path is a collection of functional units such as arithmetic logic units or multipliers that perform data processing operations, registers, and buses.

The five classic components of a computer



The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A bright blue speech bubble is centered on the page, containing the title text. The bubble has a small tail pointing downwards.

Technologies for Building Processors and Memory

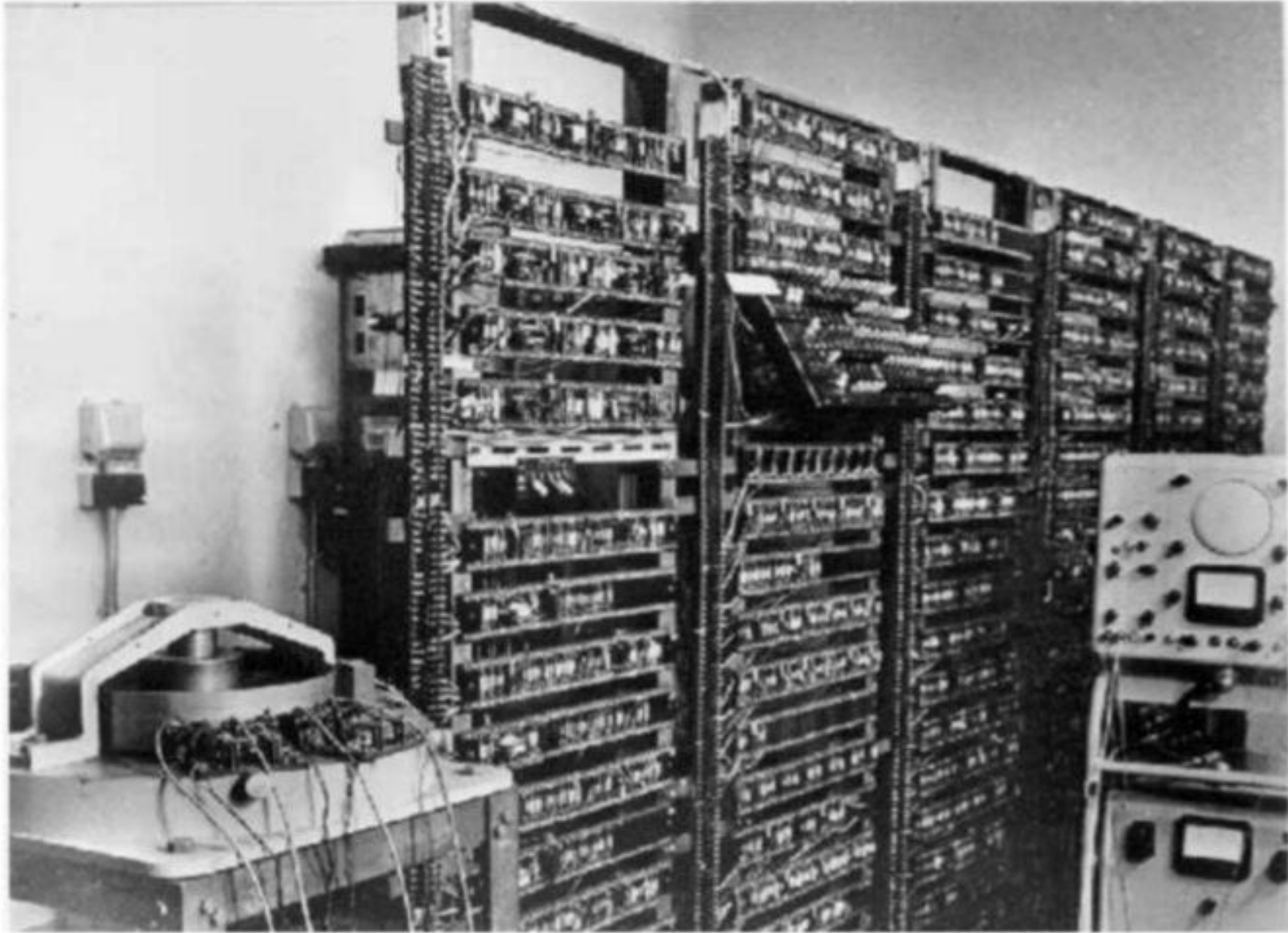
Technologies for Building Processors and Memory

- Processors and memory have improved at an incredible rate, because computer designers have long embraced the latest in electronic technology to try to win the race to design a better computer.

Year	Technology used in computers	Relative performance/unit cost
1951	Vacuum tube	1
1965	Transistor	35
1975	Integrated circuit	900
1995	Very large-scale integrated circuit	2,400,000
2013	Ultra large-scale integrated circuit	250,000,000,000

Vacuum Tubes





First Generation 1946-1959

Transistor





Second Generation 1959-1965

Integrated Circuit





Third Generation 1965-1971

VLSI





Fourth Generation 1971-1980

ULSI





Fifth Generation 1980- Present

Technologies for Building Processors and Memory

- A **transistor** is simply an on/off switch controlled by electricity.
- The *integrated circuit* (IC) combined dozens to hundreds of transistors into a single chip.
- When Gordon Moore predicted the continuous doubling of resources, he was predicting the growth rate of the number of transistors per chip.
- To describe the tremendous increase in the number of transistors from hundreds to millions, the adjective *very large scale* is added to the term, creating the abbreviation *VLSI*, for **very large-scale integrated circuit**.

Technologies for Building Processors and Memory

- This rate of increasing integration has been remarkably stable. The manufacture of a chip begins with **silicon**, a substance found in sand.
- Because silicon does not conduct electricity well, it is called a **semiconductor**.
- With a special chemical process, it is possible to add materials to silicon that allow tiny areas to transform into one of three devices:
 - Excellent conductors of electricity (using either microscopic copper or aluminum wire)
 - Excellent insulators from electricity (like plastic sheathing or glass)
 - Areas that can conduct or insulate under special conditions (as a switch)

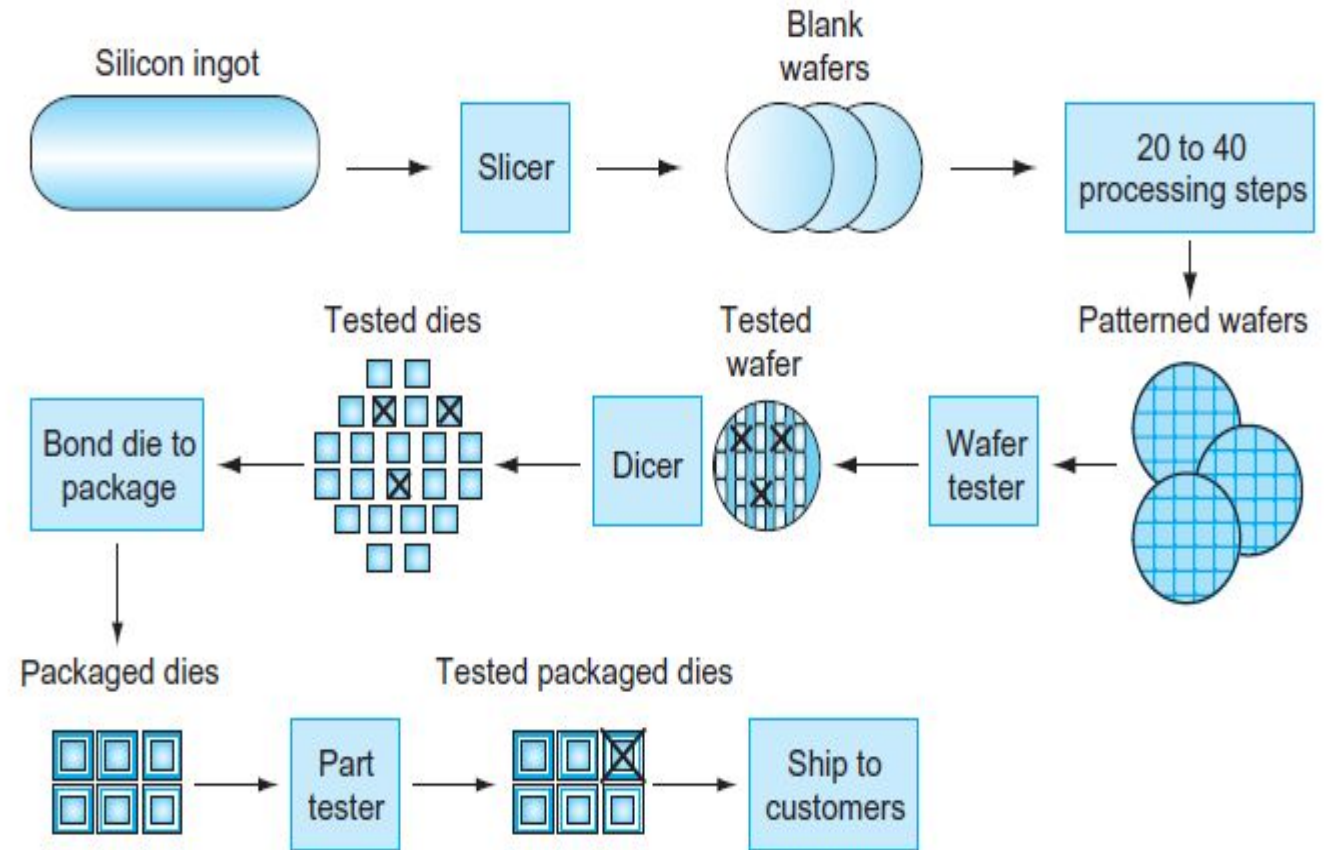
Technologies for Building Processors and Memory

- Transistors fall in the last category.
- A VLSI circuit, then, is just billions of combinations of conductors, insulators, and switches manufactured in a single small package.
- The manufacturing process for integrated circuits is critical to the cost of the chips and hence important to computer designers

Technologies for Building Processors and Memory

- The process starts with a **silicon crystal ingot**.
- Today, ingots are 8–12 inches in diameter and about 12–24 inches long.
- An ingot is finely sliced into **wafers** no more than 0.1 inches thick.
- These wafers then go through a series of processing steps, during which patterns of chemicals are placed on each wafer, creating the transistors, conductors, and insulators discussed earlier.
- Today's integrated circuits contain only one layer of transistors but may have from two to eight levels of metal conductor, separated by layers of insulators.

Technologies for Building Processors and Memory



Silicon crystal ingot

A rod composed of a silicon crystal that is between 8 and 12 inches in diameter and about 12 to 24 inches long.

Wafer

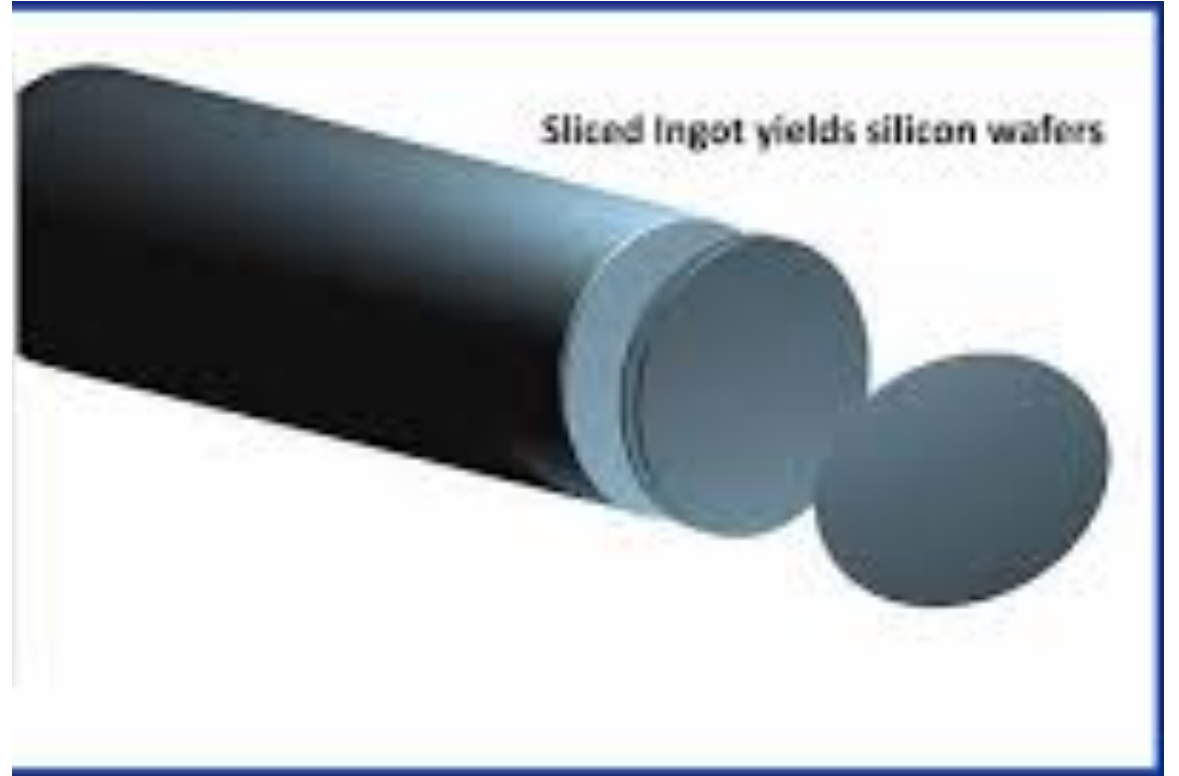
A slice from a silicon ingot no more than 0.1 inches thick, used to create chips.

- 
- 1. Purification: Raw silicon (from quartz) is purified into electronic-grade silicon.
 - 2. Crystal Growth: Using the Czochralski process, it's formed into a single crystal ingot.
 - 3. Wafer Slicing: The ingot is sliced into thin wafers.
 - 4. Photolithography: Complex patterns of transistors and circuits are etched onto the wafer.
 - 5. Doping: Impurities are added to control conductivity.
 - 6. Packaging: The wafer is cut into individual chips and packaged for use in devices.

Silicon Ingot



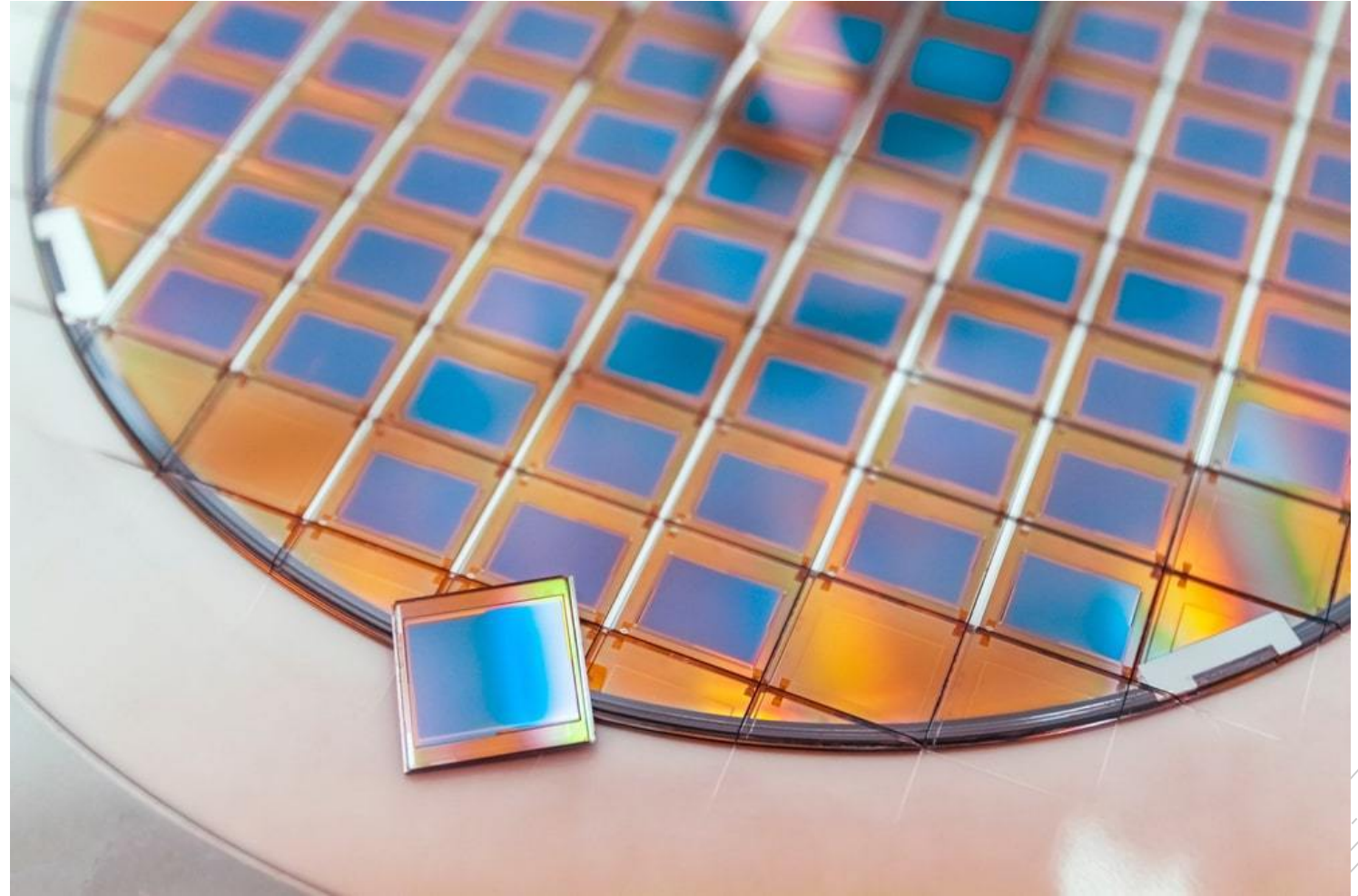
Slicing



Technologies for Building Processors and Memory

- A single microscopic flaw in the wafer itself or in one of the dozens of patterning steps can result in that area of the wafer failing.
- These **defects**, as they are called, make it virtually impossible to manufacture a perfect wafer.
- The simplest way to cope with imperfection is to place many independent components on a single wafer.
- The patterned wafer is then chopped up, or *diced*, into these components, called **dies** and more informally known as **chips**.

Wafer Dicing



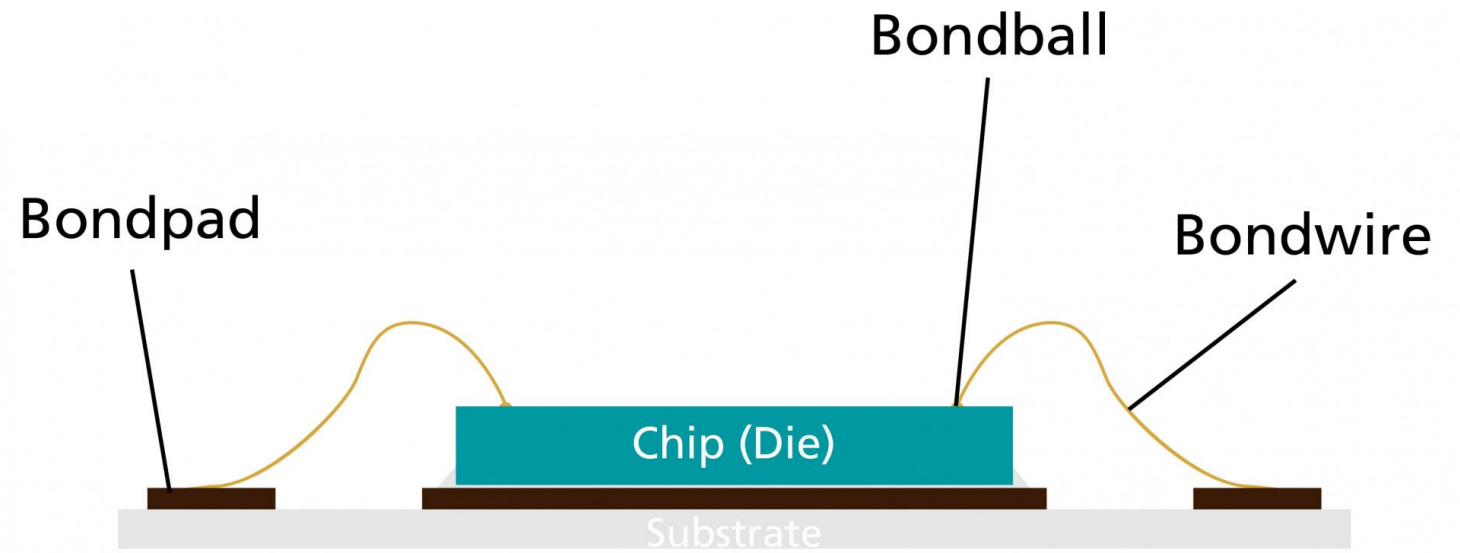
Technologies for Building Processors and Memory

- Dicing enables you to discard only those dies that were unlucky enough to contain the flaws, rather than the whole wafer.
- This concept is quantified by the **yield** of a process, which is defined as the percentage of good dies from the total number of dies on the wafer.
- The cost of an integrated circuit rises quickly as the die size increases, due both to the lower yield and the smaller number of dies that fit on a wafer.
- To reduce the cost, using the next generation process shrinks a large die as it uses smaller sizes for both transistors and wires.
- This improves the yield and the die count per wafer.

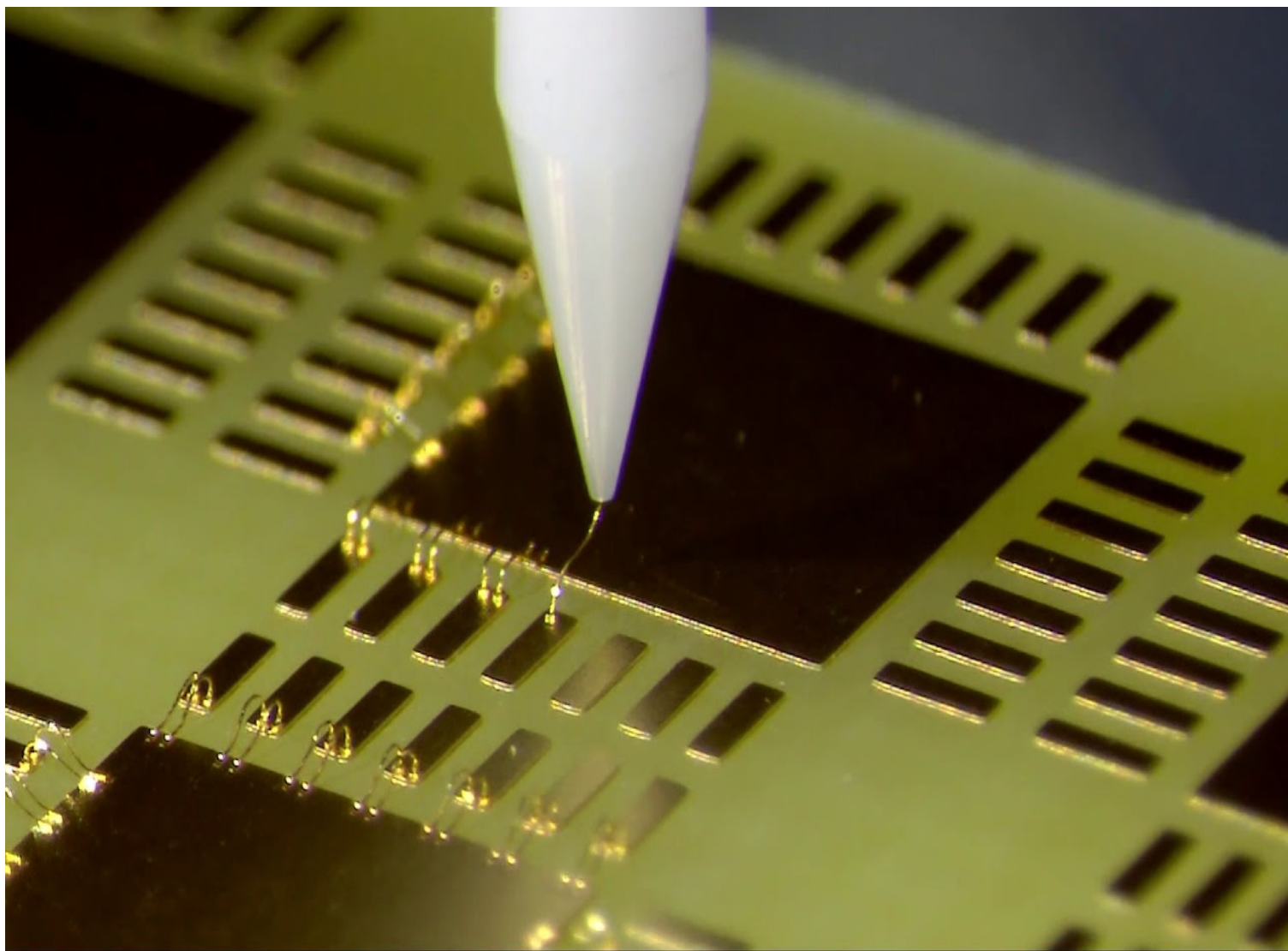
Technologies for Building Processors and Memory

- A 32-nanometer (nm) process was typical in 2012, which means essentially that the smallest feature size on the die is 32 nm.
- Once you've found good dies, they are connected to the input/output pins of a package, using a process called *bonding*.
- These packaged parts are tested a final time, since mistakes can occur in packaging, and then they are shipped to customers.

Bonding



Bonding



The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid blue speech bubble is centered on the page, pointing downwards. The word "Performance" is written in white, sans-serif font inside the bubble.

Performance

Performance

- When trying to choose among different computers, performance is an important attribute.
- Accurately measuring and comparing different computers is critical to purchasers and therefore to designers.

Performance

Defining Performance

- When we say one computer has better performance than another, what do we mean?
- If you were running a program on two different desktop computers, you'd say that the faster one is the desktop computer that gets the job done first.
- If you were running a datacenter that had several servers running jobs submitted by many users, you'd say that the faster computer was the one that completed the most jobs during a day.

Performance

- As an individual computer user, you are interested in reducing **response time**—the time between the start and completion of a task—also referred to as **execution time**.
- Datacenter managers are often interested in increasing **throughput or bandwidth**—the total amount of work done in a given time.
- Hence, in most cases, we will need different performance metrics as well as different sets of applications to benchmark personal mobile devices, which are more focused on response time, versus servers, which are more focused on throughput.

Performance

- Do the following changes to a computer system increase throughput, decrease response time, or both?
 1. Replacing the processor in a computer with a faster version
 2. Adding additional processors to a system that uses multiple processors for separate tasks—for example, searching the web
- Decreasing response time almost always improves throughput.
- Hence, in case 1, both response time and throughput are improved.
- In case 2, no one task gets work done faster, so only throughput increases.

Performance

- If, however, the demand for processing in the second case was almost as large as the throughput, the system might force requests to queue up.
- In this case, increasing the throughput could also improve response time, since it would reduce the waiting time in the queue. Thus, in many real computer systems, changing either execution time or throughput often affects the other.

Performance

- To maximize performance, we want to minimize response time or execution time for some task. Thus, we can relate performance and execution time for a computer X:

$$\text{Performance}_X = \frac{1}{\text{Execution time}_X}$$

This means that for two computers X and Y, if the performance of X is greater than the performance of Y, we have

$$\begin{aligned}\text{Performance}_X &> \text{Performance}_Y \\ \frac{1}{\text{Execution time}_X} &> \frac{1}{\text{Execution time}_Y} \\ \text{Execution time}_Y &> \text{Execution time}_X\end{aligned}$$

- That is, the execution time on Y is longer than that on X, if X is faster than Y.

Performance

- We often want to relate the performance of two different computers quantitatively.
- We will use the phrase “X is n times faster than Y”—or equivalently “X is n times as fast as Y”—to mean

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = n$$

If X is n times as fast as Y, then the execution time on Y is n times as long as it is on X:

$$\frac{\text{Performance}_X}{\text{Performance}_Y} = \frac{\text{Execution time}_Y}{\text{Execution time}_X} = n$$

If computer A runs a program in 10 seconds and computer B runs the same program in 15 seconds, how much faster is A than B?

EXAMPLE

We know that A is n times as fast as B if

$$\frac{\text{Performance}_A}{\text{Performance}_B} = \frac{\text{Execution time}_B}{\text{Execution time}_A} = n$$

Thus the performance ratio is

$$\frac{15}{10} = 1.5$$

and A is therefore 1.5 times as fast as B.

In the above example, we could also say that computer B is 1.5 times *slower than* computer A, since

$$\frac{\text{Performance}_A}{\text{Performance}_B} = 1.5$$

means that

$$\frac{\text{Performance}_A}{1.5} = \text{Performance}_B$$

ANSWER

Measuring Performance

- Time is the measure of computer performance: the computer that performs the same amount of work in the least time is the fastest.
- Program *execution time* is measured in seconds per program.
- However, time can be defined in different ways, depending on what we count.
- The most straightforward definition of time is called *response time, or elapsed time*.
- These terms mean the total time to complete a task, including disk accesses, memory accesses, *input/output* (I/O) activities, operating system overhead—everything.

Measuring Performance

- Computers are often shared, however, and a processor may work on several programs simultaneously.
- In such cases, the system may try to optimize throughput rather than attempt to minimize the elapsed time for one program.
- Hence, we often want to distinguish between the elapsed time and the time over which the processor is working on our behalf.
- **CPU execution time** or simply **CPU time**, which recognizes this distinction, is the time the CPU spends computing for this task and does not include time spent waiting for I/O or running other programs.

Measuring Performance

- CPU time can be further divided into the CPU time spent in the program, called **user CPU time**, and the CPU time spent in the operating system performing tasks on behalf of the program, called **system CPU time**.
- Differentiating between system and user CPU time is difficult to do accurately, because it is often hard to assign responsibility for operating system activities to one user program rather than another and because of the functionality differences among operating systems.

Measuring Performance

- For consistency, we maintain a distinction between performance based on elapsed time and that based on CPU execution time.
- We will use the term *system performance* to refer to elapsed time on an unloaded system and *CPU performance* to refer to user CPU time.

Measuring Performance

- Although as computer users we care about time, when we examine the details of a computer it's convenient to think about performance in other metrics.
- In particular, computer designers may want to think about a computer by using a measure that relates to how fast the hardware can perform basic functions.
- Almost all computers are constructed using a clock that determines when events take place in the hardware.

Measuring Performance

- These discrete time intervals are called **clock cycles** (or **ticks**, **clock ticks**, **clock periods**, **clocks**, **cycles**).
- Designers refer to the length of a **clock period** both as the time for a complete *clock cycle* (e.g., 250 picoseconds, or 250 ps) and as the *clock rate* (e.g., 4 gigahertz, or 4 GHz), which is the inverse of the clock period.

CPU Performance and Its Factors

- Users and designers often examine performance using different metrics.
- If we could relate these different metrics, we could determine the effect of a design change on the performance as experienced by the user.
- Since we are confining ourselves to CPU performance at this point, the bottom-line performance measure is CPU execution time.

CPU Performance and Its Factors

- A simple formula relates the most basic metrics (clock cycles and clock cycle time) to CPU time:

$$\text{CPU execution time for a program} = \text{CPU clock cycles for a program} \times \text{Clock cycle time}$$

Alternatively, because clock rate and clock cycle time are inverses,

$$\text{CPU execution time for a program} = \frac{\text{CPU clock cycles for a program}}{\text{Clock rate}}$$

- This formula makes it clear that the hardware designer can improve performance by reducing the number of clock cycles required for a program or the length of the clock cycle.

CPU Performance and Its Factors

Instruction Performance

- The performance equations above did not include any reference to the number of instructions needed for the program.
- However, since the compiler clearly generated instructions to execute, and the computer had to execute the instructions to run the program, the execution time must depend on the number of instructions in a program.

CPU Performance and Its Factors

- One way to think about execution time is that it equals the number of instructions executed multiplied by the average time per instruction.
- Therefore, the number of clock cycles required for a program can be written as

$$\text{CPU clock cycles} = \text{Instructions for a program} \times \text{Average clock cycles per instruction}$$

CPU Performance and Its Factors

- The term **clock cycles per instruction**, which is the average number of clock cycles each instruction takes to execute, is often abbreviated as **CPI**.
- Since different instructions may take different amounts of time depending on what they do, CPI is an average of all the instructions executed in the program.

CPU Performance and Its Factors

The Classic CPU Performance Equation

- We can now write this basic performance equation in terms of **instruction count** (the number of instructions executed by the program), CPI, and clock cycle time:

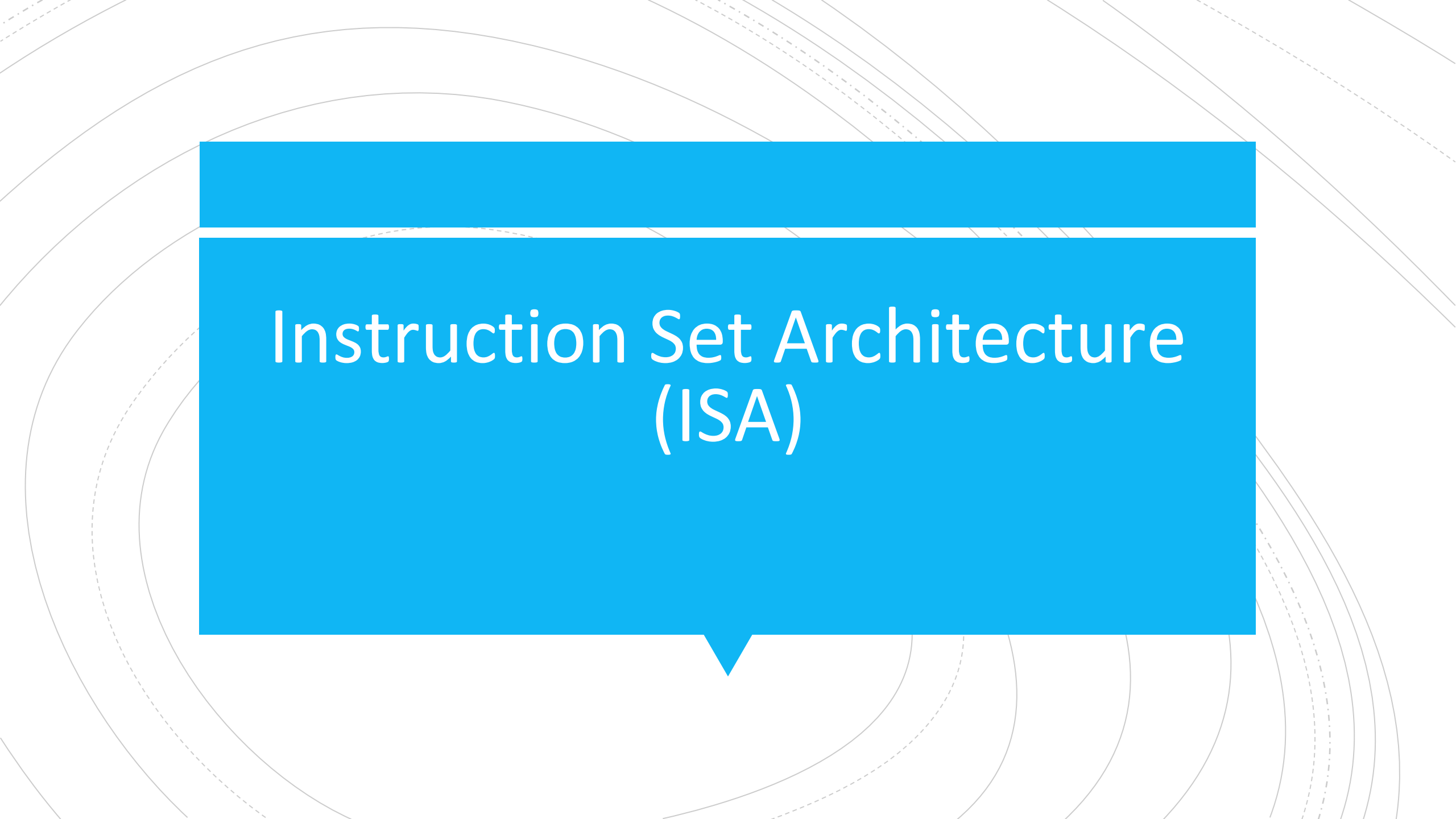
$$\text{CPU time} = \text{Instruction count} \times \text{CPI} \times \text{Clock cycle time}$$

- or, since the clock rate is the inverse of clock cycle time:

$$\text{CPU time} = \frac{\text{Instruction count} \times \text{CPI}}{\text{Clock rate}}$$

The Classic CPU Performance Equation

Components of performance	Units of measure
CPU execution time for a program	Seconds for the program
Instruction count	Instructions executed for the program
Clock cycles per instruction (CPI)	Average number of clock cycles per instruction
Clock cycle time	Seconds per clock cycle

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid blue speech bubble is centered on the page, pointing downwards.

Instruction Set Architecture (ISA)

Instruction Set

- To command a computer's hardware, you must speak its language.
- The words of a computer's language are called *instructions*, and its vocabulary is called an **instruction set**.
- The computer architecture course consists of two components – the instruction set architecture and the computer organization itself.
- The ISA specifies what the processor is capable of doing and how it gets accomplished.

Instruction Set Architecture

- So the instruction set architecture is basically the interface between your hardware and the software.
- The only way that you can interact with the hardware is the instruction set of the processor.

Instruction Set Architecture

- ISA is the portion of the machine which is visible to either the assembly language programmer or a compiler writer or an application programmer.
- It is the only interface that you have, because the instruction set architecture is the specification of what the computer can do and the machine has to be fabricated in such a way that it will execute whatever has been specified in your ISA.

Instruction Set Architecture

- The chosen instruction set comes from MIPS (Microprocessor without Interlocked Pipelined Stages) Technologies, and is an elegant example of the instruction sets designed since the 1980s.
- Every computer must be able to perform arithmetic.
- The MIPS assembly language notation
add a, b, c
- instructs a computer to add the two variables b and c and to put their sum in a.

Instruction Set Architecture

- ISAs differ based on the internal storage in a processor.
- Accordingly, the ISA can be classified as follows, based on where the operands are stored and whether they are named explicitly or implicitly:
 - **Single accumulator organization**
 - **Stack organization**
 - **General register organization**

Instruction Set Architecture

- **Single accumulator organization**, which names one of the general purpose registers as the accumulator and uses it to necessarily store one of the operands. This indicates that one of the operands is implied to be in the accumulator and it is enough if the other operand is specified along with the instruction.

Instruction Set Architecture

- **Stack organization**, where the operands are put into the stack and the operations are carried out on the top of the stack. The operands are implicitly specified here.
- **General register organization**, which specifies all the operands explicitly. Depending on whether the operands are available in memory or registers, it can be further classified as

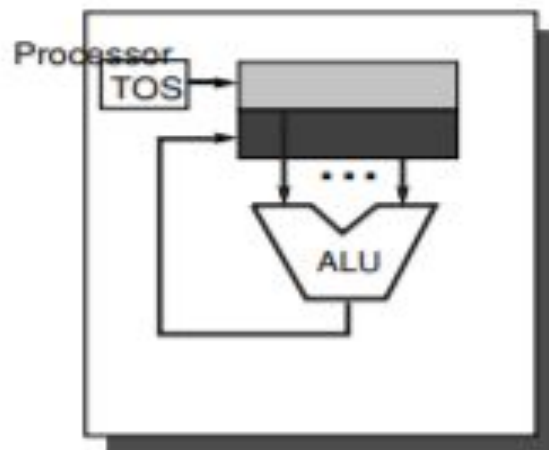
Instruction Set Architecture

- **Register – register**, where registers are used for storing operands. Such architectures are in fact also called **load – store** architectures, as only load and store instructions can have memory operands.
- **Register – memory**, where one operand is in a register and the other one in memory.
- **Memory – memory**, where all the operands are specified as memory operands.

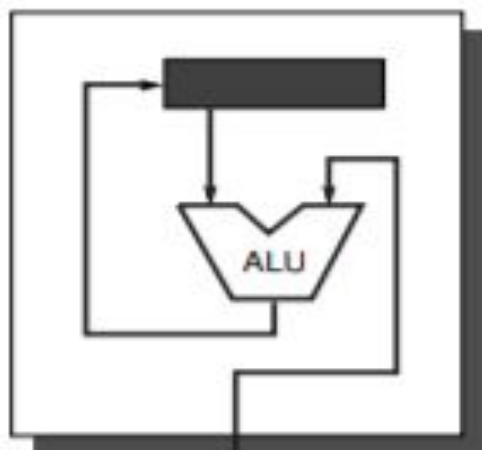
Classifying Instruction Set Architectures

- Operands may be named explicitly or implicitly.
- The operands in a stack architecture are implicitly on the top of the stack, and in an accumulator architecture one operand is implicitly the accumulator.
- The general-purpose register architectures have only explicit operands—either registers or memory locations

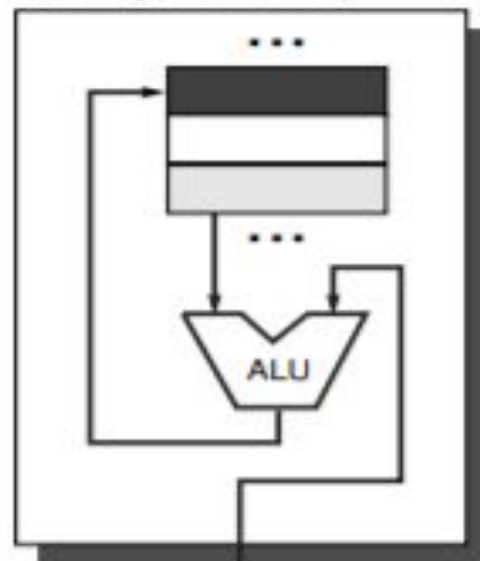
(a) Stack



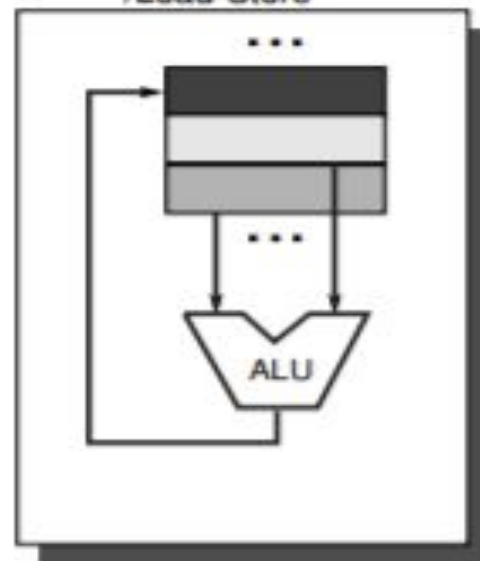
(b) Accumulator



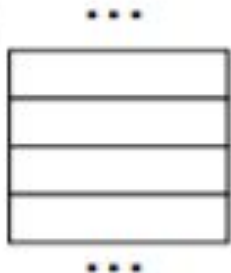
(c) Register-Memory



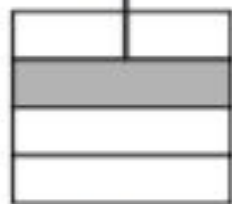
(d) Register-Register /Load-Store



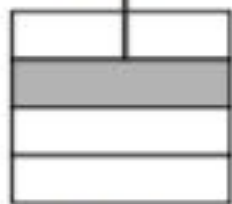
Memory



...



...



...



...

Classifying Instruction Set Architectures

Stack	Accumulator	Register (register-memory)	Register (load-store)
Push A	Load A	Load R1,A	Load R1,A
Push B	Add B	Add R3,R1,B	Load R2,B
Add	Store C	Store R3,C	Add R3,R1,R2
Pop C			Store R3,C

$$C = A + B$$

Classifying Instruction Set Architectures

- Although most early computers used stack or accumulator-style architectures, virtually every new architecture designed after 1980 uses a load-store register architecture.
- The major reasons for the emergence of general-purpose register (GPR) computers are two fold.
 - First, registers—like other forms of storage internal to the processor—are faster than memory.
 - Second, registers are more efficient for a compiler to use than other forms of internal storage.

Classifying Instruction Set Architectures

- For example, on a register computer the expression $(A*B) - (B*C) - (A*D)$ may be evaluated by doing the multiplications in any order, which may be more efficient because of the location of the operands or because of pipelining concerns.
- Nevertheless, on a stack computer the hardware must evaluate the expression in only one order, since operands are hidden on the stack, and it may have to load an operand multiple times

Classifying Instruction Set Architectures

- More importantly, registers can be used to hold variables.
- When **variables are allocated to registers**, the **memory traffic reduces**, the program speeds up (since registers are faster than memory),
- **The code density improves** (since a register can be named with fewer bits than can a memory location).

Classifying Instruction Set Architectures

- Compiler writers would prefer that all registers be equivalent and unreserved.
- Older computers compromise this desire by dedicating registers to special uses, effectively decreasing the number of general-purpose registers.
- If the number of truly general-purpose registers is too small, trying to allocate variables to registers will not be profitable.
- Instead, the compiler will reserve all the uncommitted registers for use in expression evaluation.

Classifying Instruction Set Architectures

- Most compilers reserve some registers for expression evaluation, use some for parameter passing, and allow the remainder to be allocated to hold variables.
- Just as people tend to be bigger than their parents, new instruction set architectures tend to have more registers than their ancestors.

Classifying Instruction Set Architectures

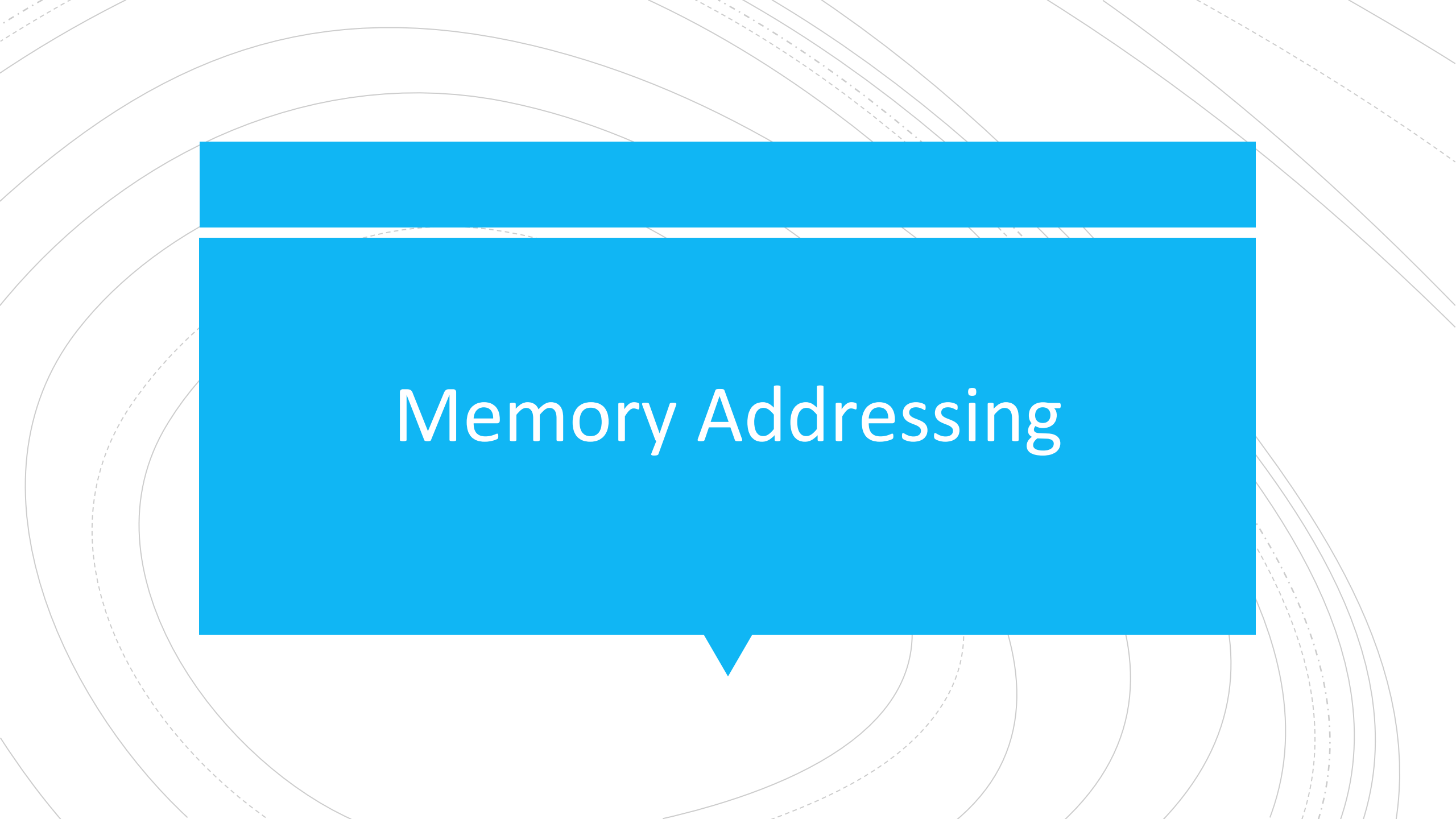
- Two major instruction set characteristics divide GPR architectures.
- Both characteristics concern the nature of operands for a typical arithmetic or logical instruction (ALU instruction).
- The first concerns whether an ALU instruction has two or three operands.
- In the three-operand format, the instruction contains one result operand and two source operands.
- In the two-operand format, one of the operands is both a source and a result for the operation.

Classifying Instruction Set Architectures

- The second distinction among GPR architectures concerns **how many of the operands may be memory addresses in ALU instructions.**
- The number of memory operands supported by a typical ALU instruction may vary from none to three

Number of memory addresses	Maximum number of operands allowed	Type of architecture	Examples
0	3	Register-register	Alpha, ARM, MIPS, PowerPC, SPARC, SuperH, Trimedia TM5200
1	2	Register-memory	IBM 360/370, Intel 80x86, Motorola 68000, TI TMS320C54x
2	2	Memory-memory	VAX (also has three-operand formats)

Type	Advantages	Disadvantages
Register-register (0,3)	Simple, fixed-length instruction encoding. Simple code-generation model. Instructions take similar numbers of clocks to execute (see App. A).	Higher instruction count than architectures with memory references in instructions. More instructions and lower instruction density leads to larger programs.
Register-memory (1,2)	Data can be accessed without a separate load instruction first. Instruction format tends to be easy to encode and yields good density.	Operands are not equivalent since a source operand in a binary operation is destroyed. Encoding a register number and a memory address in each instruction may restrict the number of registers. Clocks per instruction vary by operand location.
Memory-memory (2,2) or (3,3)	Most compact. Doesn't waste registers for temporaries.	Large variation in instruction size, especially for three-operand instructions. In addition, large variation in work per instruction. Memory accesses create memory bottleneck. (Not used today.)

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large blue speech bubble is centered on the page, containing the text 'Memory Addressing'.

Memory Addressing

Memory Addressing

- The architecture must define how memory addresses are interpreted and how they are specified.
- Memory consists of storage cells, each of which can store one bit of information(0 or 1)
- Memory is organized so that a group of n bits can be stored or retrieved in a single basic operation.
- Independent of whether the architecture is register-register or allows any operand to be a memory reference, it must define how memory addresses are interpreted and how they are specified.

Memory Addressing

- Each group of n bits is referred to as a **word** of information, and n is called **word length**.
- Modern computers have word length ranges from 16 to 64 bits.
- Accessing the memory to store or retrieve a single item of information , requires **distinct addresses** for each item location.

Memory Addressing

- Interpreting Memory Addresses
 - bytes (8 bits),
 - half words (16 bits), and
 - words (32 bits).
- Most of the computers also provide access for double words (64 bits).

Memory Addressing

A d d r e s s e s	0xFFFFFFFF	1000 0000
		...
		...
	0x00000008	0100 1001
	0x00000007	1100 1100
	0x00000006	0110 1110
	0x00000005	0110 1110
	0x00000004	0000 0000
	0x00000003	0110 1011
	0x00000002	0101 0001
	0x00000001	1100 1001
	0x00000000	0100 1111

Main Memory

Byte addressability

- Most practical assignment is to have successive address refer to successive byte locations in the memory (byte addressable memory)
- Byte locations have addresses 0,1,2,.....
- If the word length of the machine is 32 bits, successive words are located at the addresses 0,4,8, With each consisting of 4 bytes.

Memory Addressing

BIG ENDIAN AND LITTLE ENDIAN

- There are two different conventions for ordering the bytes within a larger object.
- *Little Endian* byte order puts the byte whose address is “x...x000” at the least-significant position in the double word (the little end).
- The bytes are numbered:

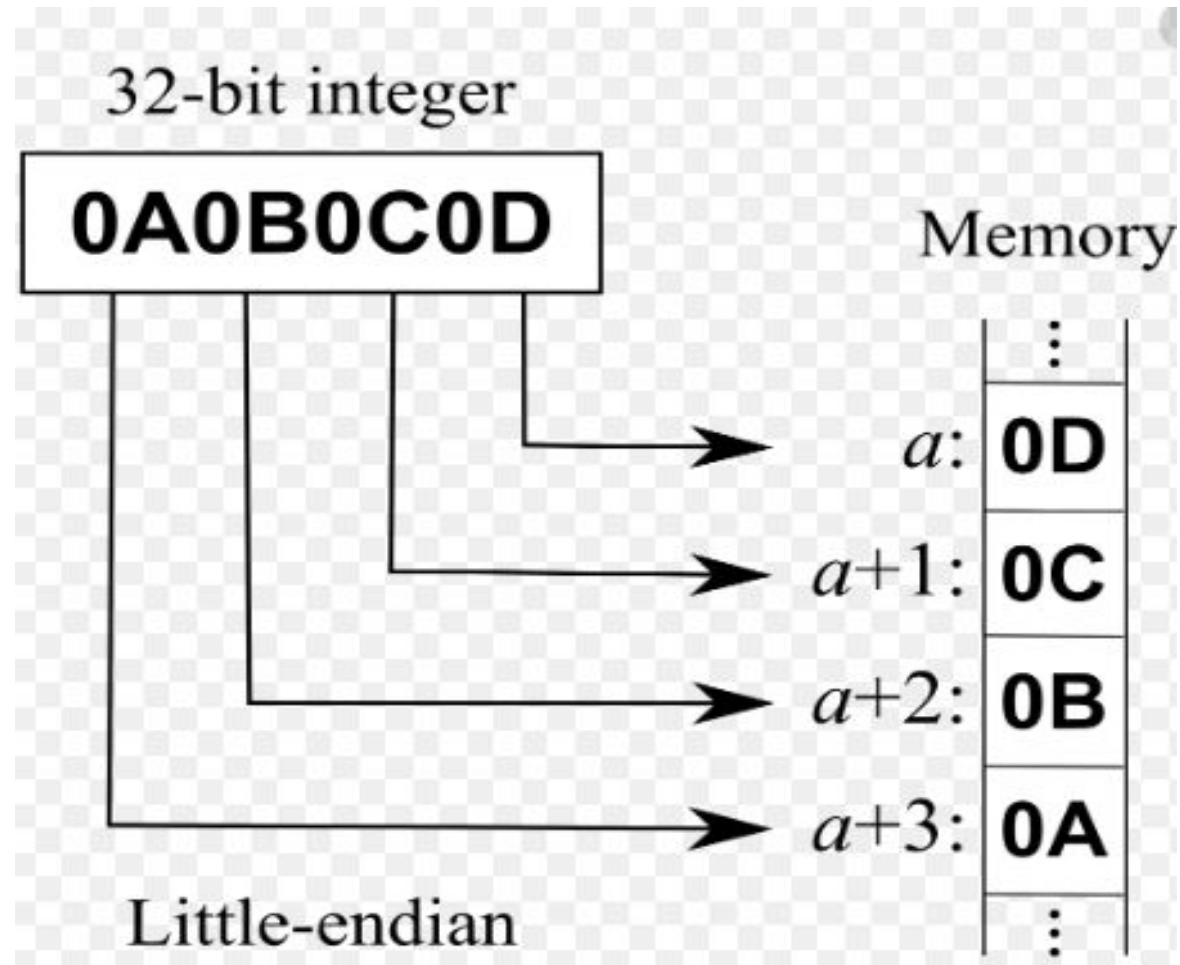


E.g.: x86, VAX



- Big-endian: Stores the most significant byte (MSB) at the lowest memory address.
- • Little-endian: Stores the least significant byte (LSB) at the lowest memory address.

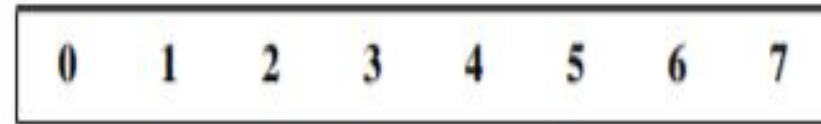
Little Endian



Memory Addressing

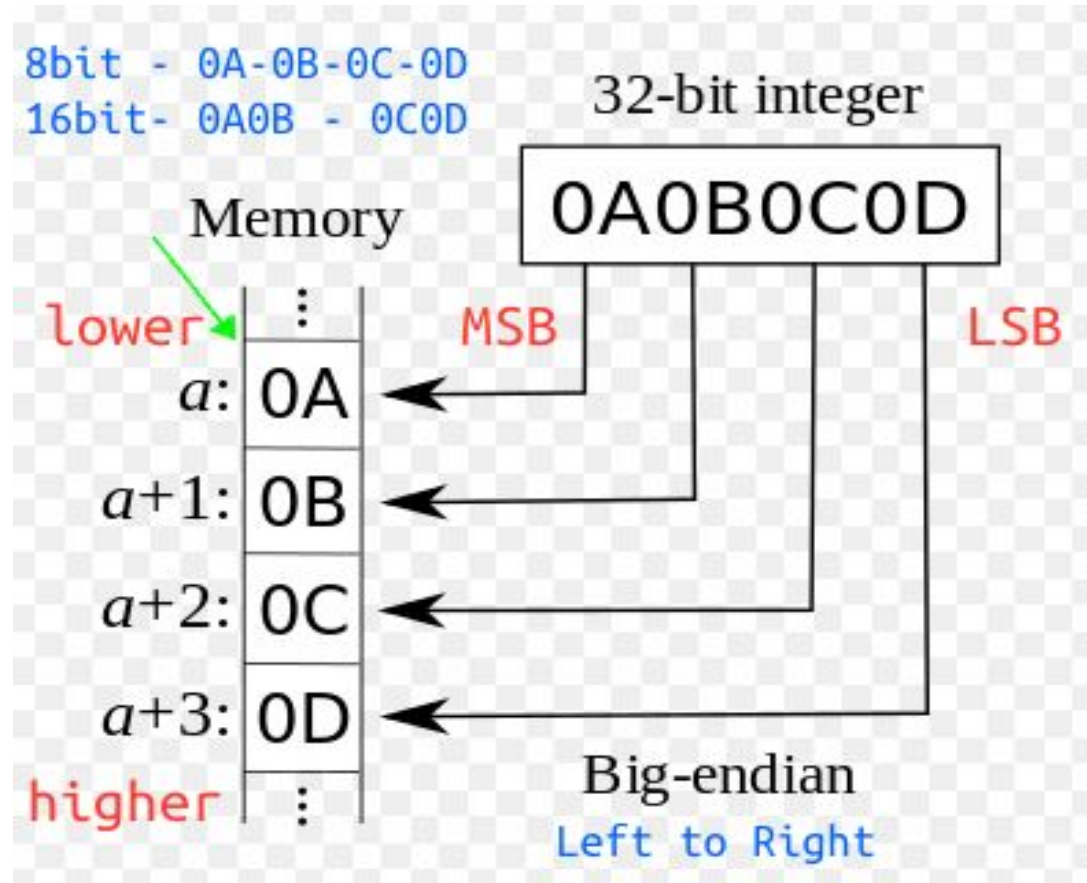
BIG ENDIAN AND LITTLE ENDIAN

- *Big Endian* byte order puts the byte whose address is “x...x000” at the most-significant position in the double word (the big end). The bytes are numbered:



- E.g.: ARM, PowerPC ,MIPS

Big Endian



Addressing Modes

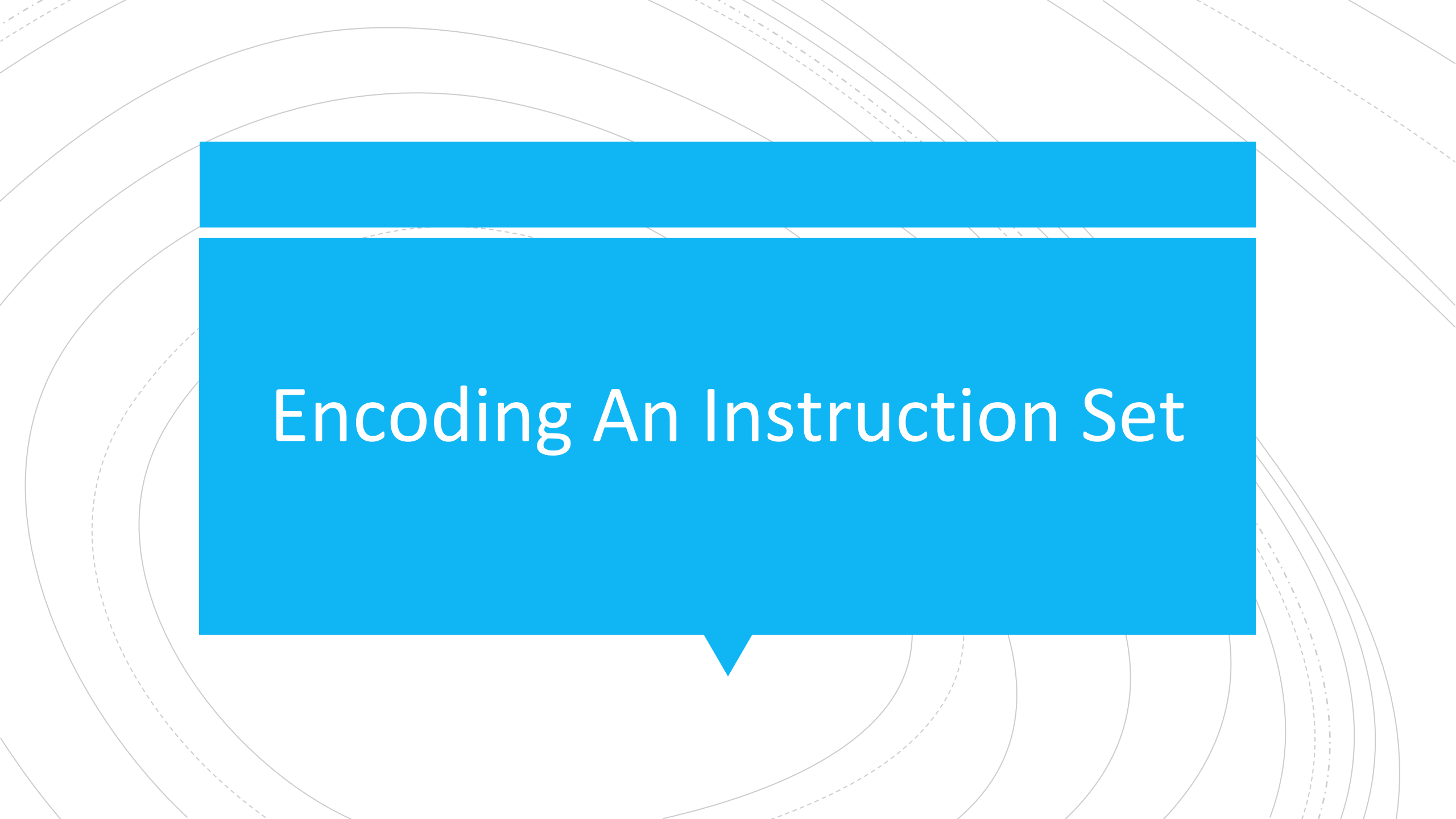
- Programs are written in high level language, which enables the programmer to use constants, local and global variables, pointers and arrays.
- When translating a high level language program to assembly language , the compiler must be able to implement these constructs using the facilities provided in the instruction set of the computer in which the program will be run.

Addressing Modes

- The different ways in which the location of an operand is specified in an instruction are referred as **addressing modes**.
- It specifies the address of an object they will access- it specifies constants and registers in addition to locations in memory.
- When a memory location is used, the actual memory address specified by the addressing mode is called the **effective address**.

Addressing mode	Example instruction	Meaning	When used
Register	Add R4, R3	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Regs}[\text{R3}]$	When a value is in a register.
Immediate	Add R4, #3	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + 3$	For constants.
Displacement	Add R4, 100 (R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[100 + \text{Regs}[\text{R1}]]$	Accessing local variables (+ simulates register indirect, direct addressing modes)
Register indirect	Add R4, (R1)	$\text{Regs}[\text{R4}] \leftarrow \text{Regs}[\text{R4}] + \text{Mem}[\text{Regs}[\text{R1}]]$	Accessing using a pointer or a computed address.
Indexed	Add R3, (R1 + R2)	$\text{Regs}[\text{R3}] \leftarrow \text{Regs}[\text{R3}] + \text{Mem}[\text{Regs}[\text{R1}] + \text{Regs}[\text{R2}]]$	Sometimes useful in array addressing: R1 = base of array; R2 = index amount.
Direct or absolute	Add R1, (1001)	$\text{Regs}[\text{R1}] \leftarrow \text{Regs}[\text{R1}] + \text{Mem}[1001]$	Sometimes useful for accessing static data; address constant may need to be large.

Addressing mode	Example instruction	Meaning	When used
Memory indirect	Add R1, @(R3)	$\text{Regs}[R1] \leftarrow \text{Regs}[R1] + \text{Mem}[\text{Mem}[\text{Regs}[R3]]]$	If R3 is the address of a pointer p , then mode yields $*p$.
Autoincrement	Add R1, (R2) +	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \\ \text{Regs}[R2] &\leftarrow \text{Regs}[R2] + d \end{aligned}$	Useful for stepping through arrays within a loop. R2 points to start of array; each reference increments R2 by size of an element, d .
Autodecrement	Add R1, -(R2)	$\begin{aligned} \text{Regs}[R2] &\leftarrow \text{Regs}[R2] - d \\ \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[\text{Regs}[R2]] \end{aligned}$	Same use as autoincrement. Autodecrement/increment can also act as push/pop to implement a stack.
Scaled	Add R1, 100(R2) [R3]	$\begin{aligned} \text{Regs}[R1] &\leftarrow \text{Regs}[R1] + \text{Mem}[100 + \text{Regs}[R2] \\ &\quad + \text{Regs}[R3] * d] \end{aligned}$	Used to index arrays. May be applied to any indexed addressing mode in some computers.

The background features a series of concentric circles in light gray, some solid and some dashed, creating a ripple effect. A large, solid blue speech bubble is centered on the page, pointing downwards.

Encoding An Instruction Set

Encoding an Instruction Set

- The instructions are encoded into a binary representation for execution by the processor.
- This representation affects not only the size of the compiled program; it affects the implementation of the processor, which must decode this representation to quickly find the operation and its operands.
- The operation is typically specified in one field, called the **opcode**.
- The important decision is how to encode the addressing modes with the operations.

Encoding an Instruction Set

- The decision depends on the range of addressing modes and the degree of independence between opcodes and modes.
- Some older computers have one to five operands with 10 addressing modes for each operand
- For such a large number of combinations, typically a separate **address specifier** is needed for each operand: the address specifier tells what addressing mode is used to access the operand.

Encoding an Instruction Set

- At the other extreme are load-store computers with only one memory operand and only one or two addressing modes; obviously, in this case, the addressing mode can be encoded as part of the opcode.
- When encoding the instructions, **the number of registers and the number of addressing modes both have a significant impact on the size of instructions**, as the register field and addressing mode field may appear many times in a single instruction.
- For most instructions many more bits are consumed in encoding addressing modes and register fields than in specifying the opcode.

Encoding an Instruction Set

- The architect must balance several competing forces when encoding the instruction set:
- 1. The desire to have as many registers and addressing modes as possible.
- 2. The impact of the size of the register and addressing mode fields on the average instruction size and hence on the average program size.
- 3. A desire to have instructions encoded into lengths that will be easy to handle in a pipelined implementation.

Encoding an Instruction Set

- As a minimum, the architect wants instructions to be in **multiples of bytes**, rather than an arbitrary bit length.
- Many desktop and server architects have chosen to use a **fixed-length instruction** to gain implementation benefits while sacrificing average code size.
- There are three type of encoding techniques
 - Variable length
 - Fixed length
 - Hybrid

Operation & no. of operands	Address specifier 1	Address field 1	...	Address specifier n	Address field n
--------------------------------	------------------------	--------------------	-----	------------------------	--------------------

(a) Variable (e.g., VAX, Intel 80x86)

Operation	Address field 1	Address field 2	Address field 3
-----------	--------------------	--------------------	--------------------

(b) Fixed (e.g., Alpha, ARM, MIPS, PowerPC, SPARC, SuperH)

Operation	Address specifier	Address field
-----------	----------------------	------------------

Operation	Address specifier 1	Address specifier 2	Address field
-----------	------------------------	------------------------	------------------

Operation	Address specifier	Address field 1	Address field 2
-----------	----------------------	--------------------	--------------------

(c) Hybrid (e.g., IBM 360/70, MIPS16, Thumb, TI TMS320C54x)

Encoding an Instruction Set

- The first we call **variable**, since it allows virtually all addressing modes to be with all operations.
- This style is best when there are many addressing modes and operations.
- The second choice we call **fixed**, since it combines the operation and the addressing mode into the opcode.
- Often fixed encoding will have only a single size for all instructions; it works best when there are few addressing modes and operations.

Encoding an Instruction Set

- The trade-off between variable encoding and fixed encoding is **size of programs versus ease of decoding in the processor**.
- Variable tries to use as few bits as possible to represent the program, but individual instructions can vary widely in both size and the amount of work to be performed.

Encoding an Instruction Set

`add EAX,1000(EBX)`

- The name `add` means a 32-bit integer add instruction with two operands, and this opcode takes 1 byte.
- An 80x86 address specifier is 1 byte, specifying the source/destination register (EAX) and the addressing mode (displacement in this case) and base register (EBX) for the second operand.

Encoding an Instruction Set

- This combination takes one byte to specify the operands.
- When in 32-bit mode , the size of the address field is either 1 byte or 4 bytes.
- Since 1000 is bigger than 2^8 , the total length of the instruction is $1 + 1 + 4 = 6$ bytes

Encoding an Instruction Set

- Given these two poles of instruction set design of variable and fixed, the third alternative immediately springs to mind: **Reduce the variability in size** and work of the variable architecture but provide multiple instruction lengths to **reduce code size**.
- This hybrid approach is the third encoding alternative.